

CH1、ISA

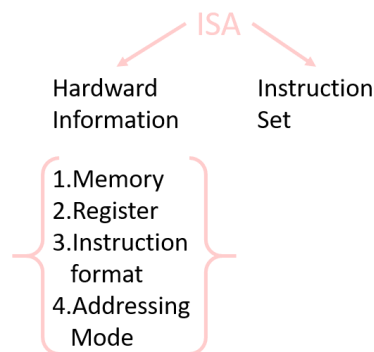
ISA 的介紹及基礎概念

重點一：基本概念

指令集(Instruction Set)：設計計算機的規格

1. 命令(Command)在高階語言中稱為敘述(Statements)、在低階語言中稱為指令(Instruction)，也是計算機可以執行的最基本運算(Primitive Operation)
2. 指令的集合為指令集(Instruction Set)·指令愈簡單、撰寫簡單、執行速度愈快、效率愈好
3. 相容性：相同指令集設計出的計算機，彼此是相容的(Compatible)，差別可能在於執行效率、能力並沒有差別

學習：指令集的規定與應用+硬體的基本規定=ISA(製作計算機的**抽象**規格：怎麼造都行，造出就好)



例(22)：Which of the following is (are) true ?

- A. Processor ISA means the instruction set architecture of the processor.
- B. ISA is an abstraction which enables different implementations of the same ISA to run identical software.
- C. For CPU time, ISA affects the CPI
- D. For CPU time, ISA affects the instruction count to use also.

A、B、C、D(C、D 第三章會提到：ISA 是唯一『都』影響效能的軟硬體 IC、CPI、Clock Rate)

重點二：MIPS(Microprocessor without Interlocked Pipe Stages)

1. 為史丹佛前校長 John L. Hennessy 發明(他同時也是 Google 創辦人的資助者)，本課程使用的為 MIPS R2000 指令集
2. CPU 一次可以處理的量叫一個：字組(word)，這個 word 若是 32bits，稱此 CPU 為 32 bits CPU；也可以從硬體的部分看出來：CPU 內暫存器。
3. 指令集可分為：精簡指令集(RISC)與複雜指令集(CISC，例子有 Intel 80x86、AMD Opteron，需死背)，1981 年前因 Compiler 技術不佳，程式員多用 CISC，以達到較輕鬆的程式設計；但 1981 年之後，隨著 Compiler 技術進步，程式員多轉以 RISC，因為其提供一個好的程式環境，並且能提高機器效率，MIPS 屬於 RISC(全世界第一個 RISC、ARM 為全世界第二個 RISC)，例(17)、例(51-2)

例(56) : About the 32-bits MIPS instructions, which description is correct ?

- A. MIPS has 32 registers inside CPU because it is a 32-bits CPU
- B. Add instruction can not directly store the addition result to memory
- C. Since memory structure is byte-addressing, the address offset in beq instruction is referred to as byte.
- D. In MIPS, “branch if less than” is realized using slt and beq/bne, since its design principle is two faster instructions are more useful than one slow and complicated instruction.

A 錯在 32 個暫存器，應為暫存器是 32 bits；B 正確，要再加上一 sw 指令；C：beq 的跳躍指令是以 label 做為目標地址

練習：由 CISC 轉 RISC 之理由

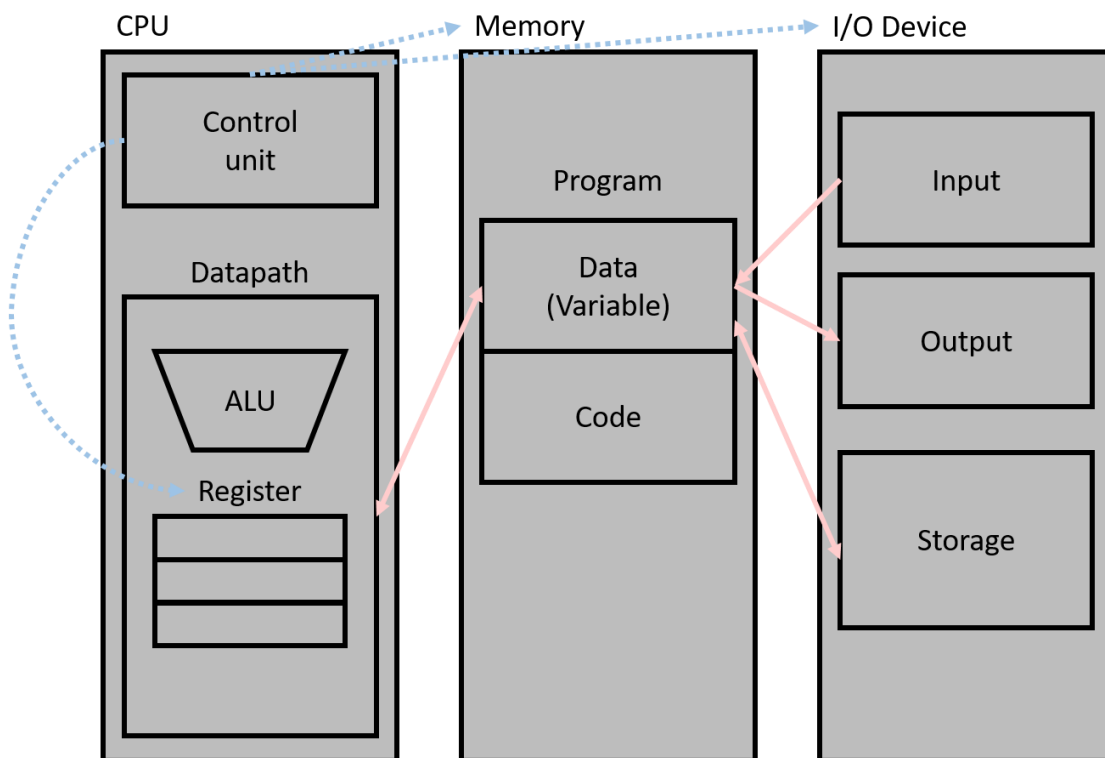
- 1. 記憶體有限：程式要能全部載入記憶體才能執行，CISC 程式較短，故使用 CISC
- 2. Compiler：Compiler 的進步使得 RISC 相對於 CISC，更容易轉譯
- 3. Pipeline：RISC 較易於 Pipeline(第四、五章會再提到)

重點三：內儲式程式概念：

現今電腦主要依兩個主要原則而建構(由匈牙利數學家提出)

- 1. 指令被表示成像數字一般的形式
 - 2. 要被執行的程式一定得先放置於記憶體中，像數字一般得被存取
- 此建構原則稱為內儲式程式概念(Stored-program 或 Von Neumann Machine)

(另一種 model 則為『量子計算機』，但目前只是純理論，無實作)



例(19) : Which of the following statements are correct ?

- A. More power instructions mean higher performance
- B. Since the commercial binary compatibility is very important, successful instruction sets don't change.
- C. Write in assembly language may not obtain the higher performance.
- D. The sequential word addresses in machines with byte addressing differ by one.

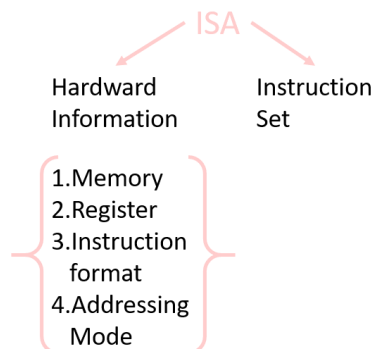
A：指令簡單才有效率；B：成功的指令集是會隨需要而改變(增減)的，例如 x86；C：正確；
D：

重點四：ISA

指令集的規定與應用+硬體的基本規定=ISA

硬體有四項：

1. 記憶體(Memory)
2. 暫存器(Register)
3. 指令格式(Instruction Format)
4. 定址模式(Addressing Mode)

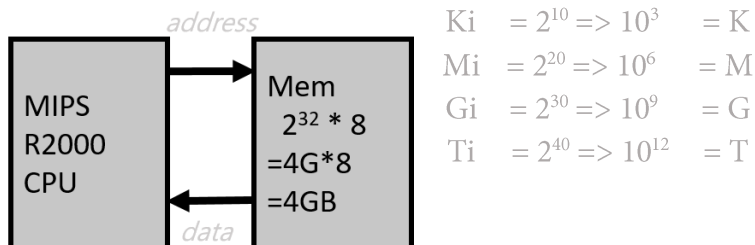


例(69-1)：Give the useful information that must be encoded in an instruction set architecture(ISA)？

1. 指令集(Instruction Set)
2. 記憶體(Memory)
3. 暫存器(Register)
4. 指令格式(Instruction Format)
5. 定址模式(Addressing Mode)

一、Memory

1. Function：存放資料，如果一個位址只能指到一個 byte 的資料量，稱為 byte address；如果一個位址只能指到一個 word 的資料量，稱為 word address
2. Structure/Addressing/Capacity(Memory 裡面永遠只放 1 個 byte/8 個 bits 的資料量)
 $Capacity = M * N = 2^k * N$



3. R/W operate：read、write 基本上互斥
4. Alignment：資料傳輸的第一個位址一定要是 4 的倍數
5. Endianness

Mem		1A	2B	3C	4D
	4N	MSB		LSB	
	4N+1				
	4N+2				
	4N+3				

Big Endianness: 1A2B3C4D，例：MIPS

Little Endianness: 4D3C2B1A，例：ARM、Intel 80x86

例(48)：翻譯：0xd75afb68 在 Big endianness 與 Little endianness 為何？

Big：d7 5a fb 68

Little：68 fb 5a d7

例(58)：if the least significant byte(B0) is stored at memory address 4N when N is an integer, this storage order is called ____ endian.

if the least significant byte(B0) is stored at memory address 4N+3 when N is an integer, this storage order is called ____ endian.

Little、Big

例(78)：翻譯：把”Tom Lien”以 little-endian 方式放入記憶體，each row consists of 4 bytes.

因為 CPU 至 Memory 的 data 寬度為 32bits=4bytes，故一個 row 可放 4bytes 即 4 個字母，且一次只傳 4 個，故 Tom_先傳、再傳 Lien，結果如下圖

	m	o	T
n	e	i	L

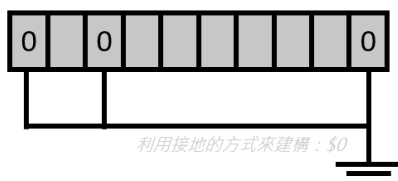
二、暫存器(Register)

1. MIPS 總共有 32 個一般目的暫存器(General Purpose Register，GPR)、32 個浮點數運算暫存器(\$f0-\$f31)、與 3 個特殊目的暫存器(Special Purpose Register，SPR：Program Counter、Hi、Lo)
2. Program Counter：存放『下一個』要被執行指令所在的記憶體位址
3. Hi、Lo：用來存乘法的運算結果、與除法的商數與餘數

32 個 GPR

Type	Number	Name	Usage
Constants	\$0	\$zero	Constant zero
Assembler Related	\$1	\$at	Preserved for Assembler
Procedure Call	\$2-\$3	\$v0, \$v1	Values for results
Procedure Call	\$4-\$7	\$a0-\$a3	Arguments(引數)
Variables/Constants	\$8-\$15	\$t0-\$t7	Temporaries
Variables/Constants	\$16-\$23	\$s0-\$s7	Save
Variables/Constants	\$24-\$25	\$t8,\$t9	More temporaries
OS Related	\$26-\$27	\$k0, \$k1	Preserved for OS
Memory Management	\$28	\$gp	Global pointer
Memory Management	\$29	\$sp	Stack pointer
Memory Management	\$30	\$fp	Frame pointer
Procedure Call	\$31	\$ra	Return address

Hardwired to zero



\$0(zero)

例(49)：The Register R0 in MIPS CPU is hardwired to zero, what is its purpose ?

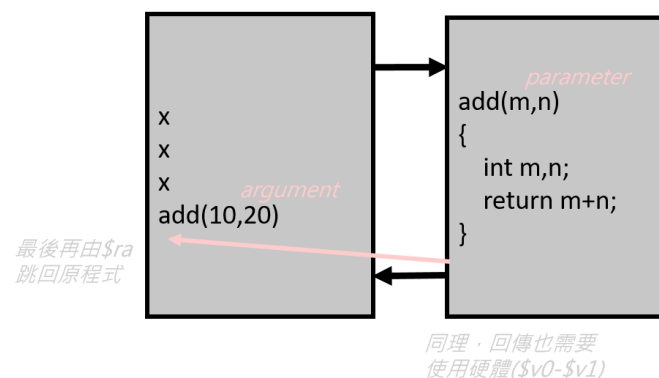
Please provide an example for its purpose.

零是非常常用的常數，例如比較大小(相減是否大於0)，因此以硬體的方式加速執行，會比由 Memory 傳值到 Register 的方式來得有效率得多

\$4-\$7(Argument)、\$2-\$3(Values for results)、\$31(Return address)

軟體(程式)之間並不會知道彼此的存在，因此當需要有 function call 傳參數時，就會需要利用到硬體來協助，類似共享記憶體的概念(只是是 *shared register*)，而共享的暫存器即為 \$a0-\$a3(\$4-\$7)，且必須依序放置，不能跳著存；同理，回傳結果值也需 \$v0-\$v1(\$2-\$3)；最後再由 \$ra(\$31)內所存的記憶體位址，跳回原程式

因軟體並不知道對方存在
故需靠硬體支援(\$a0-\$a3)



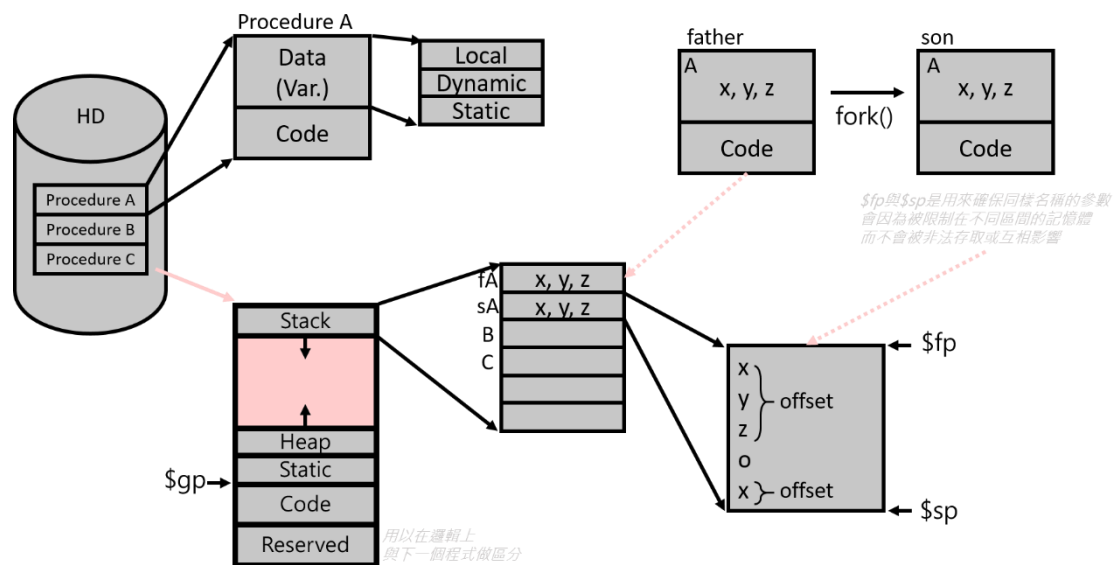
\$28-\$30(Global、Stack、Frame pointer)

一個程式在記憶體的 address space 包含了 4 或 5 個區域(考慮是否將 Reserved 列入)，由下至上為：Reserved(用以區分不同程式之間的硬體空間)、Code(Text)、Static、Heap 與 Stack。

變數上則有三種：僅限該程式的 Local 變數、動態增減的 Dynamic 變數(因為 Dynamic 變數的存放方式過於複雜，後續有提到再行解釋)、與全域共用的 Static 變數。

程序框或啟動紀錄(Procedure frame or activation record)：

其中全域共用的 Static 變數是直接存放在 Static data 裡，並以 Global Pointer(代碼&28的\$gp)來指向其位址；Local 變數則是由上往下、存放在 Stack 內，並且使用 Stack Pointer(最頂端)與 Frame Pointer(最底端)指向『正在執行的 Procedure』的變數區間位址(也就是指\$fp與\$sp會依執行的 Procedure 不同而移動)，以避免不同 Procedure 有相同變數時，會互相影響與干擾的狀況發生，而 offset 可以用來做為參數存取的參考方式(一般來說只需要\$sp一個指標即可，但因為\$sp會因 Local 變數的增減移動，故 MIPS 多加入了不動的\$fp，以利於查找)



例(57-1)：What is procedure frame? (翻譯)有了\$sp，為何還需要\$fp？

A procedure frame is the segment of stack containing a procedure's saved registers and local variables.

A frame pointer is used to point to the location of the saved register and local variables for a given procedure. A stack pointer might change during the procedure, and so references to local variable in memory might have different offsets depending on where they are in the procedure, making the procedure harder to understand. Alternatively, a frame pointer offers a stable base register within a procedure for local memory references.

當程式需要使用的變數，多於 CPU Registers 時，編譯器會將最常用的變數存在暫存器，其他則放於記憶體，並利用 load/store 來傳送資料，此方法稱為『溢出暫存器 Spilling Register(為一方法，非硬體暫存器)』

註：若 CPU Registers 過少會常常需要溢出暫存器、效率差；但反之，若過多暫存器亦會使 CPU 慢下來，因為 CPU 存取暫存器也需經過解碼程序，解碼時間過長會導致 CPU clock cycle time 變長、功率消耗也較大、效率不佳，故適當的 Registers 數目才是最佳設計(MIPS 為 32 個 GPR)

例(13)：Many programs have more variables than computers have registers. How does the computer do to conquer this problem？

電腦會讓常用的變數待在 Register 內，不常用存在記憶體，需要用到時再利用 load/store 移動變數來使用，此方法稱為『溢出暫存器 Spilling Registers』

例(9)：Explain why a small number of registers in a computer will have better performance when executing programs？

CPU 使用暫存器需經過解碼，較少的暫存器數量可使解碼時間較少、CPU clock cycle time 較短，且需提供之能源也較少，故效率較好、速度較快~

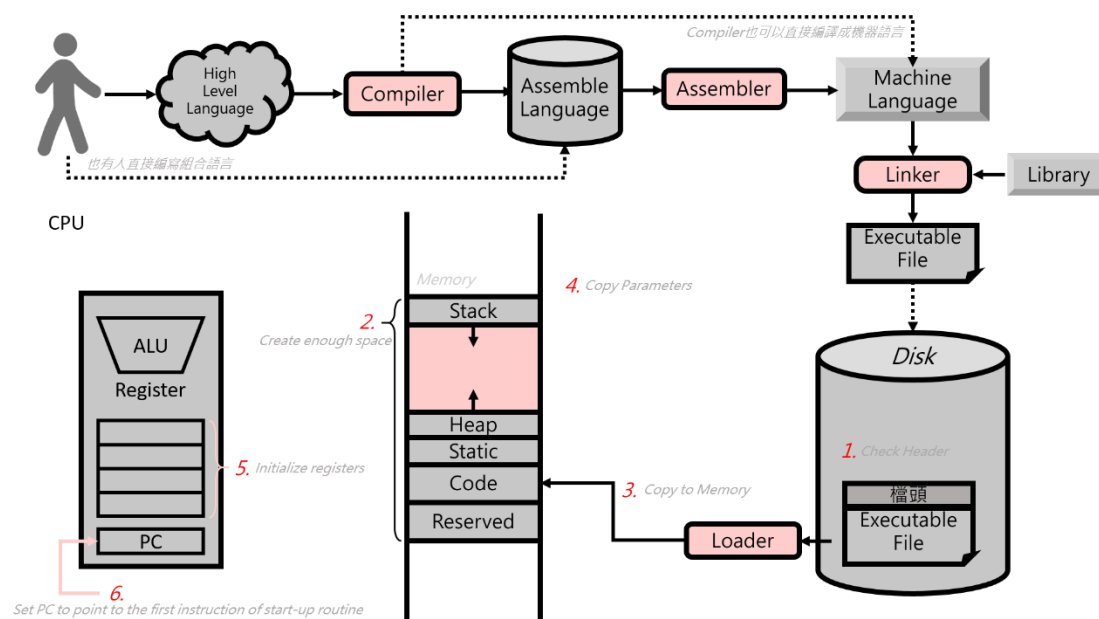
重點五：程式之轉譯與執行

1. 程式設計：一般會以高階語言(ex：C 語言)寫成，再經 Compiler 轉譯為組合語言，不過也有直接寫成組合語言的可能
2. 組合語言仍為數字及字母所組成的『符號語言』，需經過組譯器轉換成機器語言，以供硬體使用，但也有 Compiler 直接轉成機器語言的可能

- 轉換成機器語言後，若有使用到其他檔案或 Library 時，會經 Linker 一起重新排列所有執行順序，最終成為可執行的檔案

啟動程式：

- 讀取程式檔頭
- 取得足夠的記憶體空間
- 複製資料至記憶體
- 傳入參數
- 起動 CPU 內暫存器
- 設定 PC 至『啟動程式常規』的第一個指令(屬 OS，非程式本身，此舉是為了避免出破壞硬體的狀況，故一切都由 OS 來主導)



例(38)：Please describe the four steps in transforming a C program in a file on disk into a program running on a computer and show the translation hierarchy.

經 Compiler、Assembler、Linker、Loader，過程見上圖

例：Compile、Assemble、Link 與 Load 分別執行何種動作？

1. Translate a source file written in a higher level language to assembly code.	5. Jump to the main() function to start the program.
2. Turn branch labels into addresses.	6. Translate pseudo instructions to real instructions.
3. Copy the .text and .data segments from disk into memory.	7. Optimize the code.
4. For each unresolved reference to a symbol, find the definition of the symbol.	

Compile：1、7

Assemble：2、6

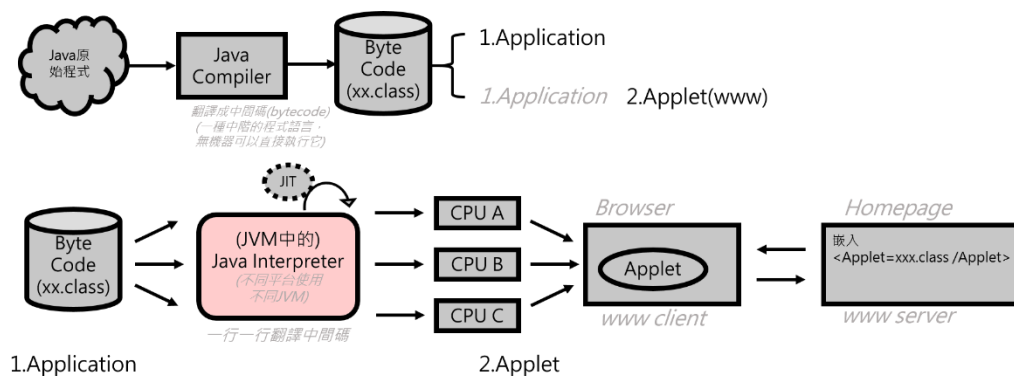
Link：4

Load：3、5

JAVA 的轉譯：

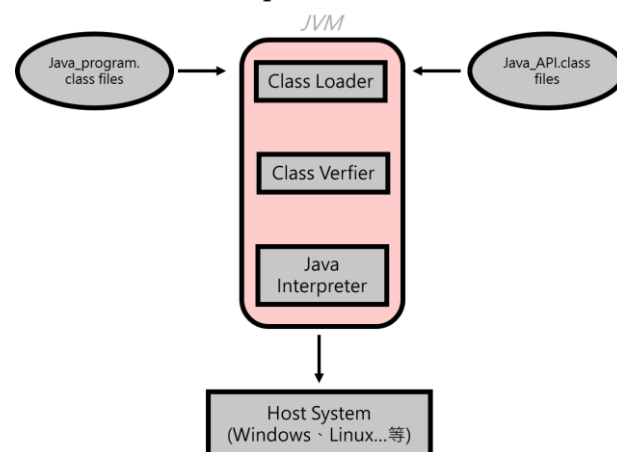
Java 是由 Sun Microsystems 於 1995 年發行，其特色為『獨立(可攜性)』、適用於多種不同平台，其流程如下：

將寫好的 Java 程式，透過 Java Compiler 編譯成 Java Bytecode(是一種中階語言，無法直接在機器上執行)，接著使用透過不同種 JVM(需下載/使用對應平台之 JVM，ex：Linux 有 Linux 自己的 JVM、Windows 有 Windows 自己的 JVM...)，將 Bytecode 一行行翻譯成該平台可用之機器語言，在不同平台上執行。但因為一行行翻譯速度較慢，故翻譯過程(可能)會使用 Just In Time Compiler(JIT)協助，JITC 會去找到最熱門的副程式，將 Bytecode 翻譯成可快速執行的機器語言。



JVM 基本上都會包含以下三種：

1. 類別載入器 class loader
2. 類別驗證器 class verifier
3. JAVA 直譯器 Java interpreter



練習：Which of the advantages of an interpreter over a translator do you think was most important for the designers of Java？

Machine Independence

例(36)：(T/F)Java programming language uses a software interpreter called JVM to translate the interpreted code segments into native code of the computer at runtime？

F：JVM 執行 Bytecode，速度很慢，故由 JITC 找到熱門副程式，轉譯成機器語言

三、ISA

Data Movement (Data Transfer)		Load(from Memory) 、store(to Memory) 、 memory-to-memory move 、 register-to-register move 、 input(from IO device) 、 output(to IO device) 、 push 、 pop(to/from stack)
Operation	Arithmetic	Integer or Flowing Point : add 、 sub 、 multiply 、 divide
	Logical	shift 、 rotate not 、 and 、 or 、 set 、 clear
Flow Control	Intra program(只在自身程式)	conditional(branch) 、 unconditional(jump)
	Inter program(跳至其他程式)	call 、 return
	System call(跳至 OS)	trap 、 return

重點七：算數指令

MIPS 的計算為二元算術運算(binary operation)，即只能將 2 個值相加，無法連加超過 3 個數值：每個指令都有 3 個運算元(operator)，順序是固定的(目的暫存器、來源暫存器 1、來源暫存器 2，運算完結果：目的暫存器會改變，但 2 個來源暫存器不會)

add 、 sub :

A	B	C	D	E	F
\$s0	\$s1	\$s2	\$s3	\$s4	\$s5

例 1：C code：A=B+C;

MIPS：

add \$s0, \$s1, \$s2

例 2：C code：A=B+C+D; E=F-A;

MIPS：

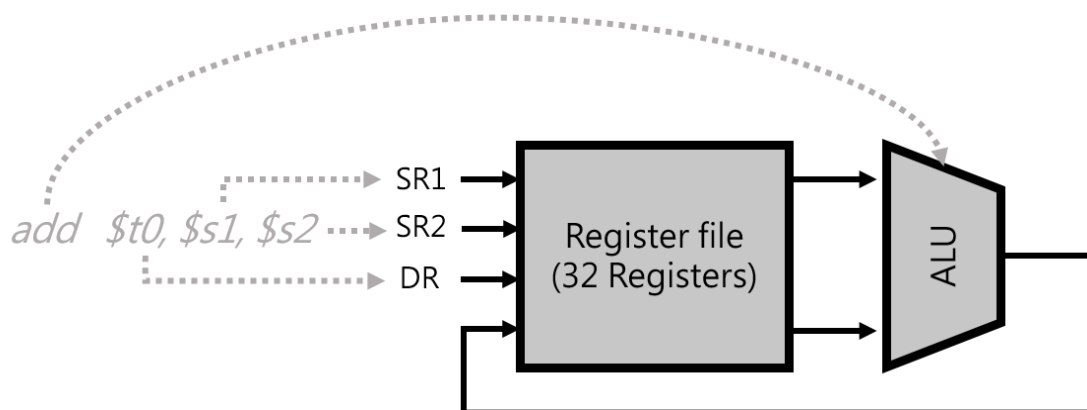
add \$t0, \$s1, \$s2

add \$s0, \$t0, \$s3

sub \$s4, \$s3, \$s0

編譯程式的三步驟：

1. 指配暫存器給予常用變數(Register assignment)：為 NP-Completed(參見演算法)
2. 轉譯(Translation)：一般來說只考慮此步驟(因 1、3 過於困難而視為已解決)
3. 最佳化(Optimization)：為 NP-Completed



例(56)：About the 32-bits MIPS instructions, which description is correct ?

- A. MIPS has 32 registers inside CPU because it is a 32-bits CPU
- B. Add instruction can not directly store the addition result to memory
- C. Since memory structure is byte-addressing, the address offset in beq instruction is referred to as byte.
- D. In MIPS, “branch if less than” is realized using slt and beq/bne, since its design principle is two faster instructions are more useful than one slow and complicated instruction.

A 錯在 32 個暫存器，應為暫存器是 32 bits；B 正確，要再加上一 sw 指令；C：beq 的跳躍指令是以 label 做為目標地址

重點八：資料轉移指令

資料轉移是指記憶體與暫存器之間的轉移，從記憶體至暫存器稱：載入(load)、反之從暫存器至記憶體稱：儲存(store)；此種方式採用『基底-位移(Base-offset)』表示方式，以避免程式無法確定參數是存取在哪一個記憶體實際位址，故以相對位址表示

load、store：

A	H
\$s0	\$s1

例 1：C code：A[12] = h + A[8];

MIPS：

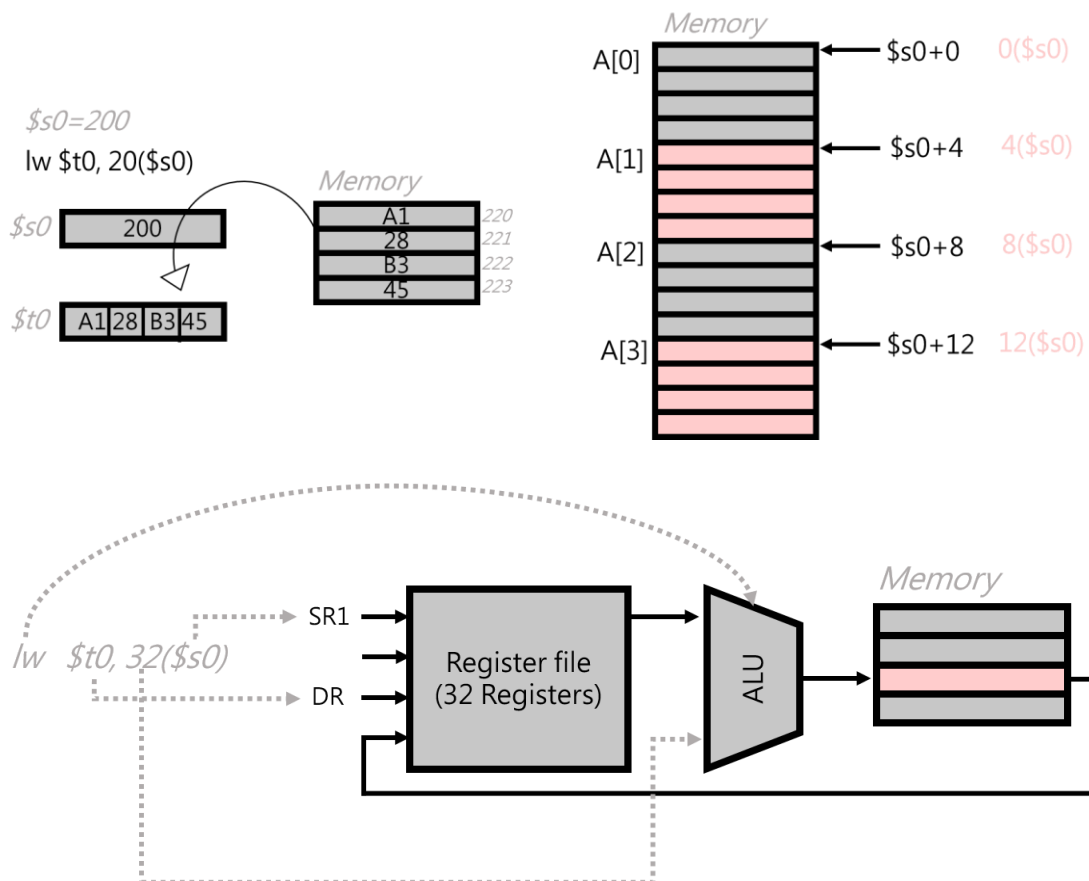
```
lw      $t0, 32($s0)
add     $t0, $s1, $t0
sw      $t0, 48($s0)
```

註：offset 在 C 語言中是以『字組(load WORD/store WORD)』的方式表示，故在組合語言中，offset 數值需乘以 4 來表示實際數目(1 word=4 bytes=32 bits=2 位元 16 進位 bits，故記憶體一格可放入 2 個數字，且 MIPS 為 Big Endianness)

1A	2B	3C	4D
----	----	----	----

MSB LSB

Mem	
1A	4N(同時也是\$fp 所指的位址)
2B	4N+1
3C	4N+2
4D	4N+3



例(24) : Write the MIPS assembly code for the C statement $A[12] = h + A[8]$; Assume that the variable h is stored in register $\$s0$, and the base address of array A is stored in $\$s1$.

```
lw    $t0, 32($s1)
add   $t0, $s0, $t0
sw    $t0, 48($s1)
```

例(61) : Assume variable h is associated with register $\$s2$ and the base address of the array A is in $\$s3$. Now the C assignment statement is as below: $A[12] = h + A[8]$; Write the compiled MIPS assembly code by filling the blanks (A) 、(B) 、(C)

```
lw    $t0, A($s3)
add   B, C, $t0
sw    $t0, 48($s3)
A : 32 、 B : $t0 、 C : $s2
```

例(68) : Which of the following is (are) data transfer instructions ?

```
jal    subroutine_1
sw     R1, 100(R2)
beq    R1, R2, start
or     R1, R2, R3
2. sw R1, 100(R2)
```

練習：(翻譯)：f=g-A[B[4]];的組合語言為何？(f, g, h, i, j 在\$s0-\$s4：A, B 在\$s6-\$s7)

```
lw    $t0, 16($s7)
sll   $t0, $t0, 2
add   $t0, $s6, $t0
lw    $s0, 0($t0)
sub   $s0, $s1, $s0
```

練習：The table below shows the values of an array stored in memory.

a.	Address	Data	b.	Address	Data
	12	1		16	1
	8	6		12	2
	4	4		8	3
	0	2		4	4
	-	-		0	5

For the memory locations in the table above, write both C and MIPS codes, respectively, to sort the data from lowest-to-highest, placing the lowest value in the smallest memory location. Assume that the data shown represents the C variable called Array, which is an array of type int. Assume the base address of Array is stored in register \$s6.

	Address	Data	C code	MIPS code
a.	12	1	tmp=Array[3];	lw \$t0, 12(\$s6)
	8	6	Array[3]= Array[2];	lw \$t1, 8 (\$s6)
	4	4	Array[2]= Array[1];	sw \$t1, 12(\$s6)
	0	2	Array[1]= Array[0];	lw \$t1, 4(\$s6)
			Array[0]= tmp;	sw \$t1, 8(\$s6)
				lw \$t1, 0(\$s6)
				sw \$t1, 4(\$s6)
				sw \$t0, 0(\$s6)
b.	16	1	tmp=Array[4];	lw \$t0, 16(\$s6)
	12	2	Array[4]= Array[0];	lw \$t1, 0 (\$s6)
	8	3	Array[0]= tmp;	sw \$t1, 16(\$s6)
	4	4	tmp=Array[3];	sw \$t0, 0(\$s6)
	0	5	Array[3]= Array[1];	lw \$t0, 12(\$s6)
			Array[1]= tmp;	lw \$t1, 4 (\$s6)
				sw \$t1, 12(\$s6)
				sw \$t0, 4(\$s6)

例(23)：The parameters v[] and k are in argument register \$a0 and \$a1, respectively. Write the MIPS assembly codes for the following procedure. Use \$t0, \$t1 and \$t3 in your codes.

```
void swap(int v[], int k)
{
    int temp;
    temp=v[k];
    v[k]=v[k+1];
    v[k+1]=temp;
}
```

```

sll    $t0, $a1, 2
add    $t1, $t0, $a0
lw     $t0, 0($t1)
lw     $t3, 4($t1)
sw     $t0, 4($t1)
sw     $t3, 0($t1)

```

練習：Assume a 32-bit machine. Assume each of the following questions starts from the table values; that is, DO NOT use value changes from one question as propagating into future parts of the question.

Register	Value	Memory Location	Value
R1	12	12	16
R2	16	16	20
R3	20	20	24
R4	24	24	28

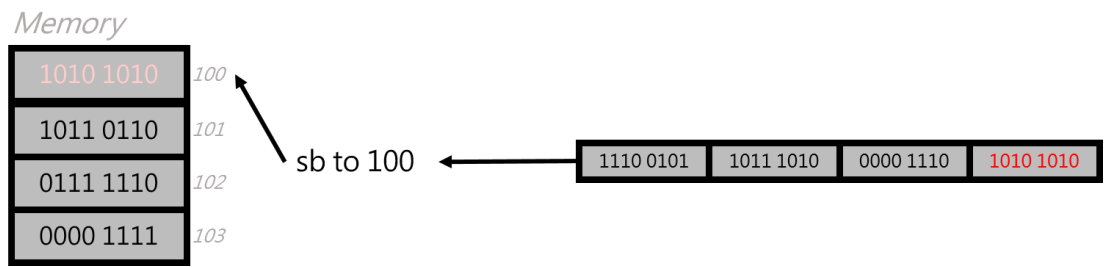
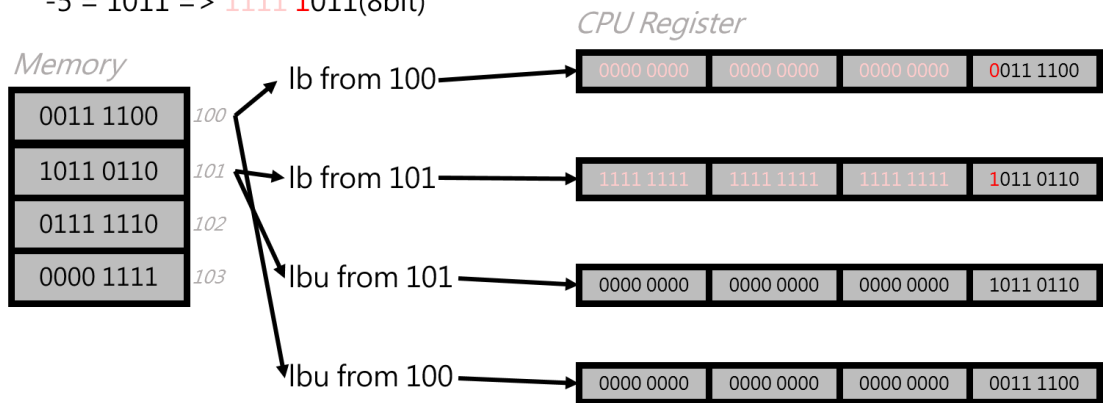
- Give the values of R1, R2, and R3 after this instruction: add R3, R2, R1
- What values will be in R1 and R3 after this instruction: load R3, 12(R1)
- What values will be in the registers after this instruction: addi R2, R3, #16

- R1=12, R2=16, R3=28
- R3=28, R1=12
- R2=36, R3=20

load 與 store 的變化(lb/lbu/sb、lh/lhu/sh)

- 與 load word 相似，lb、lh 同樣是從記憶體載入資料至暫存器，唯一不同的地方在於 byte 數：一個 word 是 4 個 bytes，故 half word 為 2 個 bytes、load byte 則為 1 個 byte；
- Signed 與 unsigned 差別在於正負號，2 補數中，最大位數(MSB)為 0 則表示正數、為 1 則表示負數，故在載入資料時，若無號數(unsigned)，空格就都填入 0 即可，但若有號數(signed)，則複製 MSB 位元至所有空位
- sb、sh 與 store word 的關係與 lb、lh 與 load word 的關係雷同，唯一差別在 lb、lh 是少位數存至多位數，會有空間要考慮；但 sb、sh 則是多位數填入少位數，故直接填完、不會有空出來的格子需要補齊，因此也沒有 signed 與 unsigned 的問題

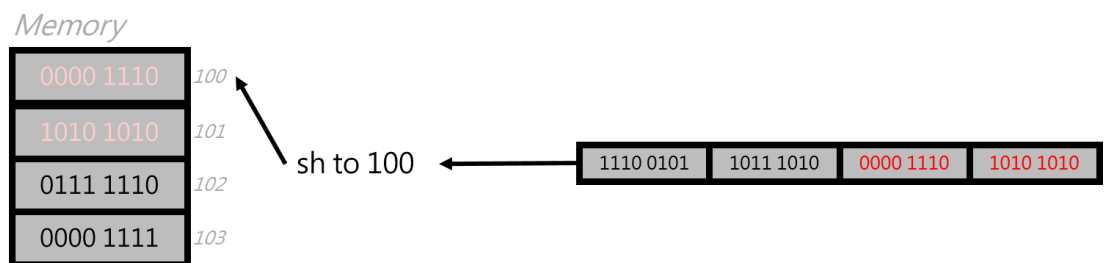
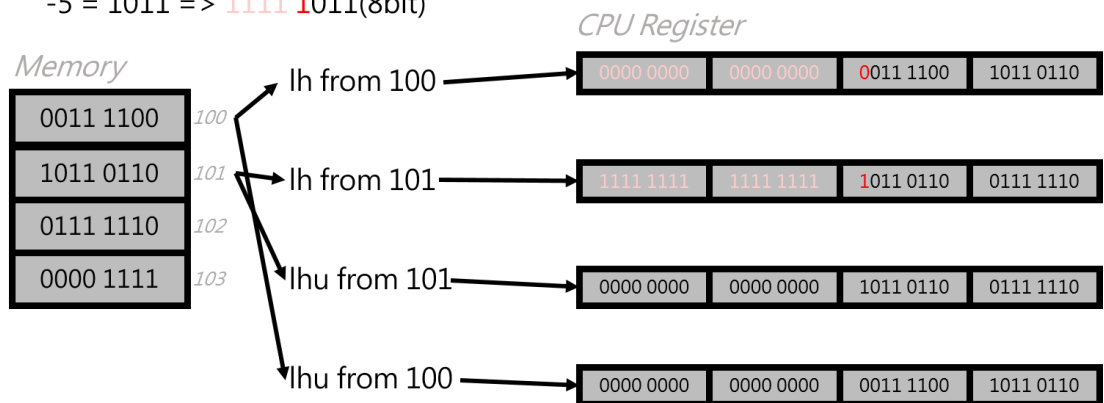
+5 = 0101 => 0000 0101(8bit)
 -5 = 1011 => 1111 1011(8bit)



lb/lbu/sb

Signed

+5 = 0101 => 0000 0101(8bit)
 -5 = 1011 => 1111 1011(8bit)



lh/lhu/sh

練習：Assume that the register \$t1 contains the address 0x100 0000 and the register \$t2 contains the address 0x1000 0010

a.	<i>lb \$t0, 0(\$t1)</i> <i>sw \$t0, 0(\$t1)</i>
b.	<i>lb \$t0, 0(\$t1)</i> <i>sb \$t0, 0(\$t2)</i>

1. Assume that the data (in hexadecimal) at address 0x1000 0000 is:

1000 0000	12	34	56	78
-----------	----	----	----	----

What value is stored at the address pointed to by register \$t2 ? Assume that the memory location pointed to \$t2 is initialized to 0xFFFF FFFF

2. Assume that the data (in hexadecimal) at address 0x1000 0000 is:

1000 0000	80	80	80	80
-----------	----	----	----	----

What value is stored at the address pointed to by register \$t2 ? Assume that the memory location pointed to \$t2 is initialized to 0x0000 0000

3. Ad

1000 0000	11	00	00	FF
-----------	----	----	----	----

What value is stored at the address pointed to by register \$t2 ? Assume that the memory location pointed to \$t2 is initialized to 0x5555 5555

	1.	2.	3.
a.	0x000 0012	0xFFFF FF80	0x0000 0011
b.	0x12FF FFFF	0x8000 0000	0x1155 5555

練習：In the snippet of MIPS assembler code below, how many times is instruction memory accessed ? How many times is data memory accessed ? (Count only accesses to memory, not registers.)

<i>lw</i>	<i>\$v1, 0(\$a0)</i>
<i>addi</i>	<i>\$v0, \$v0, 1</i>
<i>sw</i>	<i>\$v1, 0(\$a1)</i>
<i>addi</i>	<i>\$a0, \$a0, 1</i>

1. Instruction memory access: 4 times (as there are four instructions)
2. Data memory accesses: 2 times (once for the *lw* instruction and another for the *sw* instruction)

重點九：流程控制指令

流程控制指令本身並沒有執行任何運算，唯一目的只是『改變指令執行順序』。可以分為 2 類：分支(Branch)與跳躍(Jump)，而目的指令則以 Label 來代表，表示跳至要執行的記憶體位址

beq、j：

f	g	h	i	j	A
\$s0	\$s1	\$s2	\$s3	\$s4	\$s5

例 1：C code：if(i==j) goto L1; f=g+h; L1：f-i;

MIPS：

```
beq    $s3, $s4, L1
add    $s0, $s1, $s2
L1:sub    $s0, $s0, $s3
```

例 2：C code：if(i==j) f=g+h; else f=g-h;

MIPS：

```
bne    $s3, $s4, ELSE
add    $s0, $s1, $s2
j      Exit
Else: sub    $s0, $s0, $s3
Exit
```

例 3：C code：Loop g=g+A[i]; i=i+j; if(i!=h) goto Loop;

MIPS：

```
Loop: sll    $t1, $s3, 2
add    $t1, $t1, $s5
lw     $t0, 0($t1)
add    $s1, $s1, $t0
add    $s3, $s3, $s4
bne    $s3, $s2, Loop
```

例 4：MIPS：slt \$t0, \$s1, \$s2 可以表示 C code 的程式碼如下

C code：

```
if($s1 < $s2) then
    $t0=1;
else
    $t0=0;
end if
```

基本區塊(Basic Block)：

基本區塊是『編譯程式執行最佳化的最基本單位』，其定義為：一連串 1.沒有分支、及 2.沒有分支目的的指令。

1. 若有分支，也只能是區塊的『最後』一個指令
2. 若有分支目的，也只能是區塊的『第一個』指令

在程式要執行時，編譯器：

1. 會將高階語言敘述轉譯成低階語言指令
2. 再將指令分割成一個個基本區塊、並予以最佳化
3. 最後再執行各個基本區塊之間的最佳化

最佳化：

使用最少、最精簡的指令完成其程式要執行的功能、以減少程式碼佔用的記憶體空間並加速程式執行

例(75)：There are six relative conditions between the values of two registers. In this problem we consider two of them. Assuming that variable i corresponds to register \$19 and variable j to \$20, show the MIPS code for the conditions corresponding to the following two C codes:

1. if(i==j) goto L1;
 2. if(i<j) goto L1;
1. *beq \$19, \$20, L1;*
2. *slt \$at, \$19, \$20*
bne \$at, \$zero, L1

例(74)：Explain the meaning of each of the following MIPS instructions using an if-statement. Denote the program counter as PC when needed. Note that an instruction has four bytes.

slt \$s1, \$s2, \$s3
slti \$s1, \$s2, 100
bne \$s1, \$s2, 25

1. *if(\$s2 < \$s3) \$1=1; else \$1=0*
2. *if(\$s2 < 100) \$1=1; else \$1=0*
3. *if(\$s1 != \$s2) goto(PC+4)+100*

例(70)：Which of the following is (are) true ?

1. A C compiler compiles a C program into assembly language program for the target machine.
2. Pseudo-instructions are instructions which are not implemented in hardware.
3. A label is a pseudo-instruction
4. Pseudo-instructions are directives in an assembly language program

1、2、4；Label 是記憶體位址

虛擬指令：

MIPS 本身並不提供 blt(branch if less than)、bgt(branch if greater than)、ble(branch if less and equal than)、bge(branch if greater and equal than)等指令，但可以用 beq、bne、slt 等指令來合成，稱之為虛擬指令(Pseudo-instructions)

虛擬指令	MIPS 指令	虛擬指令	MIPS 指令
<i>blt \$s1, \$s2, L</i>	<i>slt \$t0, \$s1, \$s2</i> <i>bne \$t0, \$zero, L</i>	<i>bgt \$s1, \$s2, L</i>	<i>slt \$t0, \$s2, \$s1</i> <i>bne \$t0, \$zero, L</i>
<i>bge \$s1, \$s2, L</i>	<i>slt \$t0, \$s1, \$s2</i> <i>beq \$t0, \$zero, L</i>	<i>ble \$s1, \$s2, L</i>	<i>slt \$t0, \$s2, \$s1</i> <i>beq \$t0, \$zero, L</i>

註：雖然嚴格上來說 MIPS 並無上述之虛擬指令，但若在組合語言內撰寫，一樣可以達到相同效果，因為組譯器會協助自動轉譯、拆解成可執行的實際指令

練習：

<i>Pseudo-instruction</i>	<i>What is accomplished</i>	<i>Solution</i>
<i>move \$t1, \$t2</i>	$\$t1 = \$t2$	<i>add \$t1, \$t2, \$zero</i>
<i>clear \$t0</i>	$\$t0 = 0$	<i>add \$t0, \$zero, \$zero</i>
<i>beq \$t1, small, L</i>	<i>if(\$t1 = small) goto L</i>	<i>addi \$at, \$zero, small</i> <i>beq \$t1, \$at, L</i>
<i>beq \$t2, big, L</i>	<i>if(\$t2 = big) goto L</i>	<i>lui \$at, upper(big)</i> <i>ori \$at, \$at, lower(big)</i> <i>beq \$t2, \$at, L</i>
<i>li \$t1, small</i>	$\$t1 = \text{small}$	<i>addi \$t1, \$zero, small</i>
<i>li \$t2, big</i>	$\$t2 = \text{big}$	<i>lui \$t2, upper(big)</i> <i>ori \$t2, \$t2, lower(big)</i>
<i>ble \$t3, \$t5, L</i>	<i>if(\$t3 <= \$t5) goto L</i>	<i>slt \$at, \$t5, \$t3</i> <i>beq \$at, \$zero, L</i>
<i>bgt \$t4, \$t5, L</i>	<i>if(\$t4 > \$t5) goto L</i>	<i>slt \$at, \$t5, \$t4</i> <i>bne \$at, \$zero, L</i>
<i>bge \$t5, \$t3, L</i>	<i>if(\$t5 >= \$t3) goto L</i>	<i>slt \$at, \$t5, \$t3</i> <i>beq \$at, \$zero, L</i>
<i>addi \$at, \$zero, small</i>	$\$t0 = \$t2 + \text{big}$	<i>lui \$at, upper(big)</i> <i>ori \$at, \$at, lower(big)</i> <i>add \$t0, \$t2, \$at</i>
<i>lw \$t5, big(\$t2)</i>	$\$t5 = \text{Memory}[\$t2 + \text{big}]$	<i>lui \$at, upper(big)</i> <i>ori \$at, \$at, lower(big)</i> <i>add \$at, \$at, \$t2</i> <i>lw \$t5, 0(\$at)</i>

練習：C has many statements for decisions and loops, while MIPS has few. Which of the following do or do not explain this imbalance ? Why ?

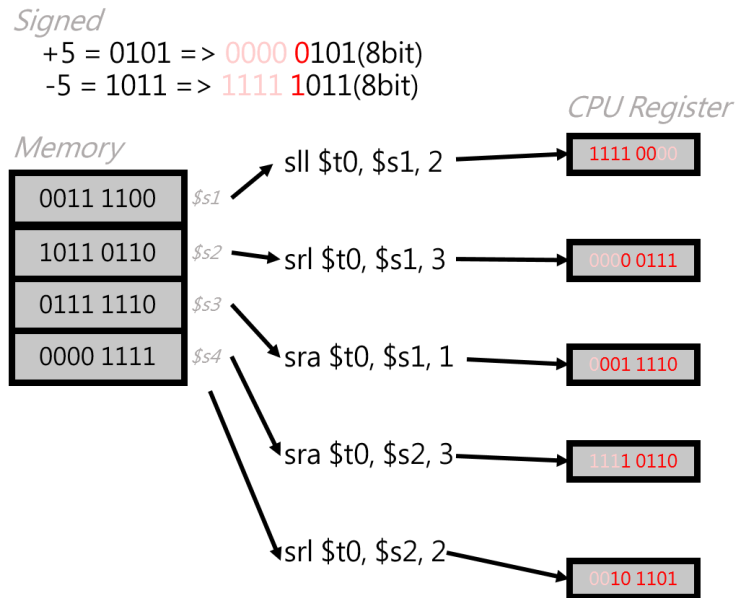
1. More decision statements make code easier to read and understand.
2. Fewer decision statements simplify the task of the underlying layer that is responsible for execution.
3. More decision statements mean fewer lines of code, which generally reduces coding time.
4. More decision statements mean fewer lines of code, which generally results in the execution of fewer operations.

全對

重點十：邏輯運算

Shift Instructions			
Logic		Arithmetic	
Left(sll)	Right(srl)	(x)Left(sla、同 sll 故不存在)	Right(sra)
Emptied bits fill with 0			Fill with sign

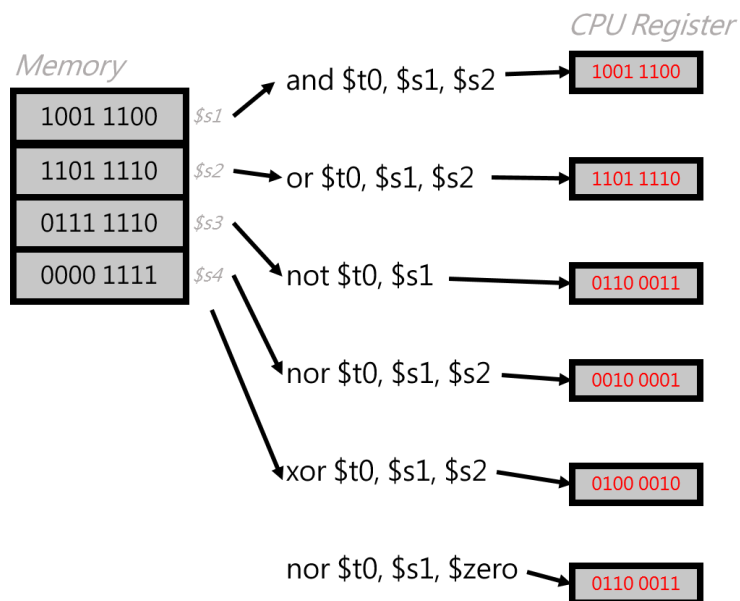
1. 雖然理論上是有 sla 的指令，但實際 MIPS 中，並不存在 sla 的指令，因為與 sll 的功能重覆，故需要用到時，直接以 sll 代替之
2. 位移量的數值為：0-31



例(41) : What does MIPS instruction srl mean ? Give you answer in one sentence.
Move all the bits of a word in a register to the right, filling the emptied bits with 0s.

And 、Or 、Not

MIPS 並不支援 Not , 但可以用 Nor 來代替(以簡化指令、增加執行速度)



例(6) : Consider two MIPS instructions: xor (exclusive OR) and nor (not OR). The table shown below defines these operations on a bit-by-bit basis.

A	B	A xor B	A nor B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	0

For each of the following new instructions, use only xor and nor to provide the minimal instruction sequence to accomplish the same thing.

1. The instruction not take the one's complement of the source register and places it in the destination register:
not \$t0, \$t1 #\$t0 = not(\$t1)
2. The instruction swap exchanges two registers. After the instruction is executed, the destination register has the original value of the source register, and the source register has the original value of the destination register:
swap \$t0, \$t1 #\$t0 <-> \$t1

1. *nor \$t0, \$t1, \$zero*
2. *xor \$t0, \$t0, \$t1*
xor \$t1, \$t0, \$t1
xor \$t0, \$t0, \$t1

例(73) : Which of the following instructions performs NOT ? (*assuming R0=0.*)

1. OR R1, R0, R3
2. AND R1, R0, R3
3. NOR R1, R0, R3
4. ADD R1, R0, R3

3

例(76) : Show the minimal MIPS instruction sequence for a new instruction called not that takes the one's complement of a Source register and places it in a Destination register. Convert this instruction (accepted by the MIPS assembler): not \$s0, \$s1 (Hint: It can be done in one instruction if you use the new logical instructions.)

Nor \$s0, \$s1, \$zero

練習 : The MIPS instruction set includes several shift instructions. They include logical-shift-left, logical-shift-right, and arithmetic-shift-right. Other architectures only provide an arithmetic-shift-left instruction.

1. Why doesn't MIPS offer an "arithmetic-shift-left" opcode ?
2. How would you implement in the assembler a logical-shift-left (LSL) pseudo-operation for a machine that didn't have this particular instruction ?

1. 因為實際執行上，sll 與 sla 功能相同，故為了精簡指令、增加效率，就刪去 sla 此一指令
2. (sll \$s1, \$s2, \$zero) can be done via the following loop:

```

    addi $t0, $zero, n
    add $s1, $s2, $zero
Loop:  beq $t0, $zero, Exit
    add $s1, $s1, $s1
    addi $t0, $t0, -1
    j Loop
Exit

```

Mask 、Set

可使用 Add 與 Or 來進行 Mask 與 Set

<i>Mask</i>	<i>Set</i>
$\$s1=1001\ 0101$	$\$s1=1001\ 0101$
$\$s2=0011\ 1100$	$\$s2=0011\ 1100$
<i>and \$s1, \$s1, \$s2</i>	<i>or \$s1, \$s1, \$s2</i>
$\$s1=0001\ 0100$	$\$s1=1011\ 1101$

練習：The logical instructions below are not included in the MIPS instruction set, but can be synthesized using one or more MIPS assembly instructions. Provide a minimal set of MIPS instructions that may be used in place of the instructions. If the value of $\$t2 = 0x00FFA5A5$ and the value of $\$t3 = 0xFFFF003C$, what is the result in $\$t1$?

a.	<i>andn \$t1, \$t2, \$3</i>	//bit-wise AND of \$t2, !\$t3
b.	<i>xnor \$t1, \$t2, \$3</i>	//bit-wise exclusive-nor

	1.	2.
a.	<i>nor \$t1, \$t3, \$t3</i> <i>and \$t1, \$t1, \$t2</i>	0x0000A581
b.	<i>xor \$t1, \$t3, \$t3</i> <i>nor \$t1, \$t1, \$t1</i>	0x00FF5A66

練習：The table below shows different C statements that use logical operators. Write a minimal sequence of MIPS assembly instructions that does the identical operation. If the memory location at C[0] contains the integer value 0x0000 1234, and the initial integer value of A and B are 0x0000 0000 and 0x0000 2222, what is the result value of A ?

a	$A = B \& C[0]$
b	$A = A \& B : C[0]$

Assuming $\$t1 = A$, $\$t2 = B$, $\$s1 = \text{base of Array C}$

	1	2
a	<i>lw \$t3, 0(\$s1)</i> <i>and \$t1, \$t2, \$t3</i>	0x0000 0220
b	<i>beq \$t1, \$zero, Else</i> <i>add \$t2, \$t2, \$z</i> <i>beq \$zero, \$zero, End</i> <i>Else: lw \$t1, 0(\$s1)</i> <i>End:</i>	0x0000 1234

練習：The operation xor stands for exclusive OR. The following table defines this operation on a bit-by-bit basis.

A	B	$A \text{ xor } B$	$A \text{ nor } B$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	0

Show the minimal MIPS instruction sequence for a new instruction called swap that exchanges two registers. After the sequence completes, the Destination register has the original value of the Source register, and the Source register has the original value of the Destination register. Convert this instruction:

```
swap $s0, $s1
```

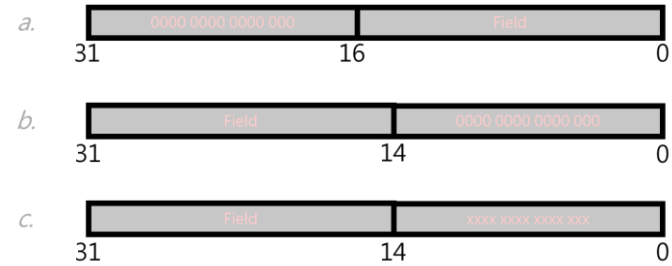
This sequence must use only these two registers. [Hint: It can be done in three instructions if you use the logical instruction xor.]

Instruction sequence		\$s0	\$s1
<code>xor \$s0, \$s0, \$s1</code>	=>	$\$s0 \oplus \$s1$	$\$s1$
<code>xor \$s1, \$s0, \$s1</code>	=>	$\$s0 \oplus \$s1$	$(\$s0 \oplus \$s1) \oplus \$s1 = \$s0$
<code>xor \$s0, \$s0, \$s1</code>	=>	$(\$s0 \oplus \$s1) \oplus \$s0 = \$s1$	$\$s0$

練習：The following figure shows the placement of a bit field in register \$t0



Find the shortest sequence of MIPS instructions that extracts a field from \$t0 for the constant values i = 22, and j = 5, and places the field into \$t1 in the format shown in the data table. The bits shown as “xxx” are to remain unchanged.



		(另解)
a	<code>sll \$t1 \$t0, 9</code> <code>srl \$t1 \$t1, 15</code>	<code>srl \$t1 \$t0, 6</code> <code>lui \$t2, 0x0001</code> <code>ori \$t2, \$t2, 0xFFFF</code> <code>and \$t1, \$t1, \$t2</code>
b	<code>srl \$t1 \$t0, 6</code> <code>sll \$t1 \$t1, 15</code>	<code>sll \$t1 \$t0, 9</code> <code>lui \$t2, 0xFFFF</code> <code>ori \$t2, \$t2, 0x8000</code> <code>and \$t1, \$t1, \$t2</code>
c	<code>srl \$t2 \$t0, 6</code> <code>srl \$t2 \$t2, 15</code> <code>sll \$t1 \$t1, 17</code> <code>srl \$t1 \$t1, 17</code> <code>or \$t1 \$t2, \$t1</code>	<code>sll \$t2 \$t0, 9</code> <code>lui \$t3, 0xFFFF</code> <code>ori \$t3, \$t3, 0x8000</code> <code>and \$t2, \$t2, \$t3</code> <code>andi \$t1, \$t1, 0x7FFF</code> <code>or \$t1, \$t1, \$t2</code>

重點十一：常數

常數在 MIPS 中也相當常用到，但若是以記憶體存取的方式執行，會過於沒效率，因此增加一些專用指令：『立即指令 Immediate Instruction』來進行常數的運算。然而因為指令格式的關係(後面會再提到)，只剩下 16-bits，故能表示的常數只 16-bits(32-bits 扣掉 6-bits 的指令、2 個 5-bits 暫存器位址(目標與來源暫存器各一))，再加上正負有號數的考量，其範圍為：負的($2^{15}-1$)到正 2^{15} ，因此在計算時需特別注意。若要將 \$t3 加 4，其指令如下：

`addi $t3, $s3, 4`

因為指令設計愈簡單愈有效率，故在立即指令中，只有加法的 `addi`，沒有減法的 `subi`，因為若要相減，只需要加上一個負值即可，例如下列 2 指令即相等：

`subi $t3, $s3, 1` 與 `addi $t3, $s3, -1`

練習：The data table contains the values for registers \$t0 and \$t1

a.	\$t0 = 0x5555 5555, \$t1 = 0x1234 5678
b.	\$t0 = 0xBEAD FEED, \$t1 = 0xDEAD FADE

For the values in the table above, what is the value of \$t2 for the following sequence of instructions:

- `sll $t2, $t0, 4`
`or $t2, $t2, $t1`
- `sll $t2, $t0, 4`
`andi $t2, $t2, -1`
- `srl $t2, $t0, 3`
`andi $t2, $t2, 0xFFEF`

註：常數只有 16 個 bits，故 $-1 = 0x0000\text{ FFFF}$

	1	2	3
a.	0x5775 5778	0x0000 5550	0x0000 AAAA
b.	0xFEFF FEDE	0x0000 EED0	0x0000 BFCD

例(64)：Please use at most one instruction to generate assembly code for each of the following C statements (assuming variable a and b are unsigned integers). You can use the variable names as the register names in your assembly code.

(註：and immediate = and rt, rs, imm)

- `b = a / 8;`
- `b = a % 16;`

- `srl b, a, 3`
- `and b, a, 15` //註：要注意本題有改 and immediate 之指令格式，故不為 `andi`，”i”需刪去

32 位元的立即運算元

因為常數的立即指令只能存取 16-bits 的數字，故當需要存取完整的 32-bits 數字時，需要用到其他指令：載入上半部立即指令(load upper immediate, lui)與 `ori`，載入方法可見下例：

例(28) : Consider the following binary data shown in the table.

a.	1010 1101 0001 0000 0000 0000 0000 0010
b.	1111 1111 1111 1111 1111 1111 1111 1111

1. Write the MIPS code that creates the 32-bits constants listed above and stores that value to register \$t1. Please use separate codes for the two cases.
2. If the current value of the PC is 0x00000060/0x00400600, can you use a single branch instruction to get to the PC address as show in the table above ?

1	a	lui \$t1, -21232 ori \$t1, \$t1, 2	
	b	addi \$t1, \$zero, -1	
2	a	NO	$0xAD10\ 0002 - 0x0000\ 0600 > 2^{17}$ (bytes)
		NO	$0xAD10\ 0002 - 0x0040\ 0600 > 2^{17}$ (bytes)
	b	NO	$0xFFFF\ FFFF - 0x0000\ 0600 > 2^{17}$ (bytes)
		NO	$0xFFFF\ FFFF - 0x0040\ 0600 > 2^{17}$ (bytes)

練習 : Why doesn't MIPS have a subtract immediate instruction ?

因為 MIPS 支援 add immediate 指令，且 immediate 欄位值可以正數或負數，所以 subtract immediate 就不需要包含於 MIPS 指令中

重點十二：組合語言與機器語言

四、指令格式(Instruction Format)

因為 MIPS 格式的 CPU 為 32-bits，故指令長度也為 32-bits，而每個 Register 是 32-bits = 2^5 -bits，所以暫存器位址則以 5 個 bits 來代替。我們可以將指令簡單分為 3 種：三個暫存器的 R type、2 個暫存器的 I type 與跳躍指令的 J type



Field	Length	Meaning Represented
op	6	因為所有的指令數少於 64 種 = 2^6 種，故以 6-bits 來表示即可
rs	5	來源暫存器 1 (First Register Source Operand)
rt	5	來源暫存器 2 (Second Register Source Operand)，s 下一個字母為 t
rd	5	目標暫存器 (Register Destination Operand)
shamt	5	位移量 (Shift amount)
funct	6	主要為 R type 之下，更細分 function 的代碼
addr/immdata	16	表示常數或位址
address	26	跳躍位址

組合語言 => 機器語言：

1. 先了解其指令是哪種 type，並找到對應的 opcode (ex : R type=0、lw=35、j=2)
2. 了解其類別後，將暫存器代碼依格式放入 rs、rt、(rd)
3. 最後將剩餘資料補上，即完成轉碼工作

機器語言 => 組合語言：

1. 先將前 6-bits 之 opcode，透過查表了解整組指令碼之格式與對應指令
2. 依其格式切出暫存器並放於組合語言對應暫存器代碼位置
3. 將剩餘數字轉換成組合語言需要之程式碼

Opcode Table

Op(31:26)								
	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	R-type	Bltz/gez	jump	jump & link	beq	bne	blez	bgtz
1(001)	addi	addiu	slti	sltiu	andi	ori	xori	lui
2(010)	TLB	Flpt						
3(011)								
4(100)	lb	lh	lwl	lw	lbu	lhu	lwr	
5(101)	sb	sh	swl	sw			swr	
6(110)	lwc0	lwc1						
7(111)	swc0	swc1						

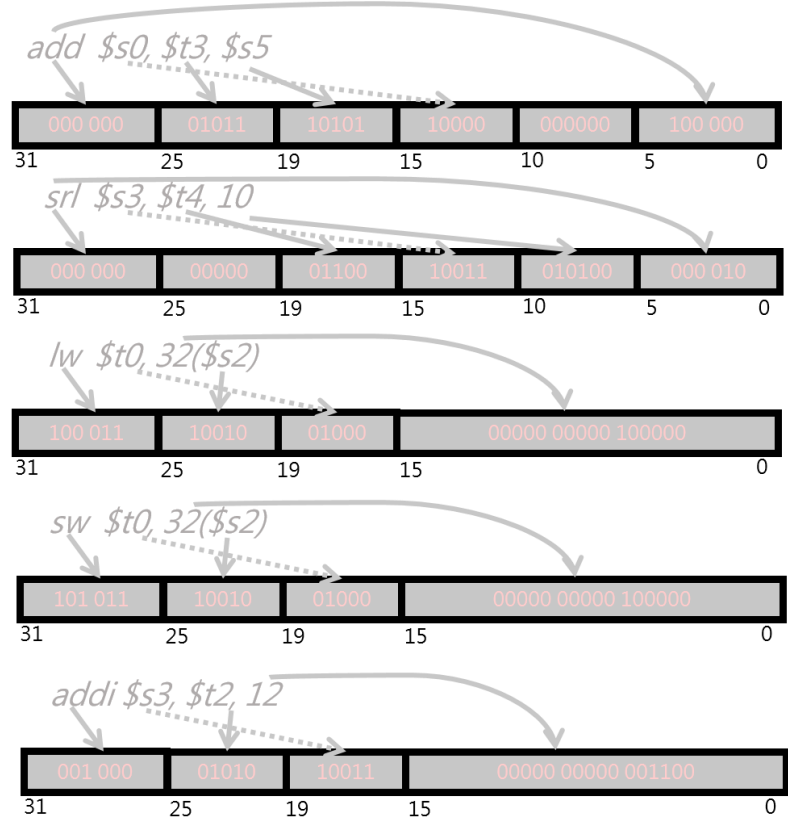
Function Code Table

Op(31:26)								
	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	<i>sll</i>		<i>srl</i>	<i>sra</i>	<i>sllv</i>		<i>srlv</i>	<i>srav</i>
1(001)	<i>jump reg</i>	<i>jalr</i>			<i>syscall</i>	<i>break</i>		
2(010)	<i>mfhi</i>	<i>mthi</i>	<i>mflo</i>	<i>mtlo</i>				
3(011)	<i>mult</i>	<i>multu</i>	<i>div</i>	<i>divu</i>				
4(100)	<i>add</i>	<i>addu</i>	<i>sub</i>	<i>subu</i>	<i>and</i>	<i>or</i>	<i>xor</i>	<i>nor</i>
5(101)			<i>slt</i>	<i>sltu</i>				
6(110)								
7(111)								

32 個 GPR

Type	Number	Name	Usage
Constants	\$0	\$zero	Constant zero
Assembler Related	\$1	\$at	Preserved for Assembler
Procedure Call	\$2-\$3	\$v0, \$v1	Values for results
Procedure Call	\$4-\$7	\$a0-\$a3	Arguments(引數)
Variables/Constants	\$8-\$15	\$t0-\$t7	Temporaries
Variables/Constants	\$16-\$23	\$s0-\$s7	Save
Variables/Constants	\$24-\$25	\$t8,\$t9	More temporaries
OS Related	\$26-\$27	\$k0, \$k1	Preserved for OS
Memory Management	\$28	\$gp	Global pointer
Memory Management	\$29	\$sp	Stack pointer
Memory Management	\$30	\$fp	Frame pointer
Procedure Call	\$31	\$ra	Return address

指令翻譯(rs、rt、rd 的位置要特別注意，依不同指令格式有不同順序)



Jump 與 beq :

Loop:	sll	\$t1, \$s3, 2	8000 0000	0	0	19	9	2	0
	add	\$t1, \$t1, \$s6	8000 0004	0	9	22	9	0	32
	lw	\$t0, 0(\$t1)	8000 0008	35	9	8	0		
	bne	\$t0, \$s5, Exit	8000 0012	5	8	21	2(向下2個指令到達Exit)		
	addi	\$s3, \$s3, 1	8000 0016	8	19	19	1		
	j	Loop	8000 0020	2	20000				
Exit:			8000 0024						

beq : 因為 branch 指令通常不會跳躍過遠(大約一半的 branch , 跳躍距離小於 16 個指令), 於是採用『PC 相對定址法(PC-relative addressing)』, 以指令數目的『位移量』來執行跳躍: 正數為向下、負數為向上, 並且是以 PC 中的數值(下一個要執行的指令位址)做為參考基準。例如上圖中的 bne, PC 值本來為 0x8000 0016, 經過 bne 完成的指令計算(PC 值+2*4=0x8000 0024), 即可跳至 Exit 的指令位址。然而雖然 branch 通常不會跳躍太遠, 但因其空間為 16-bits, 故其跳躍範圍是- 2^{15} 至 $2^{15}-1$ 字組(或- 2^{17} 至 $2^{17}-4$ 個位元組)

jump : MIPS 系統中的記憶體為 4GB, 但在管理上會將它切分為 16 個相同大小的區塊(每塊 256MB), 而任何一個程式都只能在同一個區塊內執行: 故每一個區塊的最左邊 4-bits 相同($16=2^4$, 故前 4-bits 皆相同), 又每一次的資料存取是以字組為單位, 即最右邊 2-bits 皆為 0, 所以 jump 指令只需要記錄中間 26-bits 之值(32-左邊 4-右邊 2=26), 剛好也與扣掉 6-bits opcode 後的 26-bits 相同。故 jump 採用直接定址法, 並且實際數值要經過除 4(扣去最右 2-bits)的計算

例(33) : Suppose we have a 32-bits computer with an instruction set that supports immediate instructions as shown below:



1. How many registers at most does this computer have?
2. How many operations at most can this computer have?
3. What is the range of the number in the Immediate field in 2's complement format?

1. $2^5 = 32$ registers //取 rs 與 rt 較小者
2. $2^6 = 64$ operations //opcode
3. -2^{15} to $2^{15}-1 = -32768$ to 32767

例(18) : Assume we are designing a 16-bits MIPS CPU with 16-bits instruction words. Please adjust the instruction formats according to the following specification. Assume this CPU has 6 arithmetic/logic instructions and 10 other instructions. In addition, assume there are 16 registers in this CPU; each is 16-bits long. If the 30operand format (R type) is still adopted, what are the number bits of the fields 1 2 3 in this 16-bits instruction format? Please explain your reasons.



例(2) : Given a 32-bits machine, determine the values of the labels ELSE and DONE of the following segment of instructions. Assume that the first instruction is loaded into memory location 8000_(hex)

```
8000      slt $s2, $t0, $t0
8004      bne $t2, $zero, ELSE
8008      j DONE
800C Else: addi $t2, $t2, 2
8010 DONE:
```

ELSE=800C : DONE=8010

注 : Label 是指在:之前,而不是指令最後的部分,指令最後的部分稱為 reference,而 reference 的 ELSE 與 DONE 分別應該為 : 1 與 2004_(hex)

例(34) : Assume that \$s1 and \$s2 are used for the input and both initially contain the integers, a and b, respectively. Assume that \$v0 is used for the output.

```
8000 4000h      add $t0, $zero, $zero
                loop:    beq $s2, $zero, finish
                        add $t0, $t0, $s1
                        sub $s2, $s2, 1
                        j loop
                finish:  addi $t0, $t0, 100
                        add $v0, $t0, $zero
```

1. Please describe in one mathematic formula what it computes (in terms of a, b).
2. If we assume the above code starting at location 8000 4000h in memory, and the register numbers of \$s1 and \$zero are 17 and 0, respectively. What are the MIPS machine codes for the beq (OP code is 4) and j (OP code is 2) instructions above? Please answer both in hexadecimal format.

1. The procedure computes $a*b+100$
2. 如下

Instruction	Binary format	
beq	000 100 10010 00000 00000 00000 000011	1240 0003h
j	000 010 00000 00000 00010 00000 000001	0800 1001h

例(31) : Let the value of the program counter (PC) be 0xCF01 FC00. What is the target address of the instruction, j 0x20?

```
0xC000 0080
1011 0000 0000 0000 00 0000 0010 0000 00
```

例(43) : Consider the assembly code as follows. Assume that the starting address of the first instruction is 10000 in the decimal notation. Translate the first and last instructions into corresponding machine codes, given that the opcodes of beq and j are 000100 and 000010 in binary notation.

```
Loop: beq $9, $0, End
      add $8, $8, $10
      addi $9, $9, -1
      j Loop
End:
```

beq \$9, \$0, End	000100 01001 00000 000000000000000011
j Loop	000010 00000 00000 00001 00111 000100

例(60) : While executing the MIPS code shown below, what is the target address of the branch instruction if it is taken? (Assume the starting address of this code segment is 28_{dec}.)

```
lw $4, 50($7)
beq $1, $4, 3
add $5, $3, $4
sub $6, $4, $3
or $7, $5, $2
slt $8, $5, $6
```

Branch target address will be the address of instruction *slt*; therefore, the target address = $28 + 5 * 4 = 48_{dec}$.

練習 :

a.	0x0000 1000
b.	0x2000 1400

- Suppose that register \$t0 contains a value from above. What is the value of \$t2 after the following instructions?

```
slt $t2, $t0, $t0
bne $t2, $zero, ELSE
j DONE
```

ELSE: *addi \$t2, \$t2, 2*

DONE:

- Suppose that register \$t0 contains a value from above. What is the value of \$t2 after the following instructions?

```
sll $t0, $t0, 2
slt $t2, $t0, $zero
```

- Suppose the program counter (PC) is set to 0x2000 0000. Is it possible to use the jump (j) MIPS assembly instruction to set the PC to the address as shown in the data table above? Is it possible to use the branch-on-equal (beq) MIPS assembly instruction to set the PC to the address as shown in the data table above?

	1	2	3
a.	\$t2 = 0	\$t2 = 0	Jump->no, beq->no
b.	\$t2 = 0	\$t2 = 1	Jump->yes, beq->yes

例(4) : Suppose the program counter (PC) is at address 0x0000 0000. Please answer the question below.

- Is it possible to use one single beq MIPS instruction to get to address 0x0003 0000
- Same to question 1. Except that the address is changed to 0xFFFF FF00
- Is it possible to use one jump MIPS instruction to get to address 0x0003 0000?
- Same to question 3. except that the address is changed to 0xFFFF FF00

1	2	3	4
no	yes	yes	no

例(28) : Consider the following binary data shown in the table

a.	1010 1101 0001 0000 0000 0000 0000 0010
b.	1111 1111 1111 1111 1111 1111 1111 1111

1. Write the MIPS code that creates the 32-bits constants listed above and stores that value to register \$1. Please use separate codes for the two cases.
2. If the current value of the PC is 0x0000 0600, can you use a single branch instruction to get to the PC address as shown in the table above? If the current value of the PC is 0x0040 0600, can you use a single branch instruction to get to the PC address as shown in the table above? Please give your answer for the two cases in the table.

1	a.	lui \$t1, -21232 ori \$t1, \$t1, 2	
	b.	addi \$t, \$zero, -1	
2	a.	No	0xAD10 0002 - 0X0000 0600 > 2 ¹⁷ (bytes)
		No	0xAD10 0002 - 0X0040 0600 > 2 ¹⁷ (bytes)
	b.	No	0xFFFF FFFF - 0X0000 0600 < 2 ¹⁷ (bytes)
		Yes	0xFFFF FFFF - 0X0040 0600 > 2 ¹⁷ (bytes)

練習 : What is the range of addresses for conditional branches in MIPS (K=1024)?

1. Address between 0 and 64k-1
2. Address between 0 and 256k-1
3. Addresses up to about 32k before the branch to about 32k after
4. Addresses up to about 128k before the branch to about 128k after

4

練習 : What is the range of addresses for jump and jump and link in MIPS (M=1024k)?

1. Addresses between 0 and 64M-1
2. Addresses between 0 and 64M-1
3. Addresses up to about 32M before the branch to about 32M after
4. Addresses up to about 128M before the branch to about 128M after
5. Anywhere within a block of 64M addresses where the PC supplies the upper 6 bits
6. Anywhere within a block of 256M addresses where the PC supplies the upper 4 bits

6

例(49) : Given a MIPS program with corresponding addresses below

Address16	MIPS assembly instructions
FC80 0000	L1 : add \$s0, \$s1, \$s0
...	...
FC8A 0000	bne \$s0, \$s1, L1
...	lw \$s0, 12(\$s1)

1. Is it true that the range of address for beq in MIPS is: up to about 2¹⁵ bytes before the branch to about 2¹⁵ bytes after
2. Is it true that the range of address for j in MIPS is: anywhere within block of 2²⁶ bytes addresses where the PC supplies the upper 6 bits?
3. What problem the above MIPS program has?
4. Please rewrite the code using minimal sequence so that if \$s0 is equal to \$s1, the next instruction to be executed will be the instruction at label L1.

5. Given that the register numbers of \$s0 and \$s1 are 1610 and 1710, respectively, and that the opcodes of the instructions beq, bne, slt, j, jal, lw are 4, 5, 0, 2, 3 and 35, respectively. Please convert your assembly code to MIPS machine instruction formats (using binary formats).

Address16	Machine instructions
FC80 0000	000000 10000 10001 10000 000000 000000
...	...
FC8A 0000	
FC8A 0004	
FC8A 0008	
...	...

1. No, the range should be up to about 2^{15} words before the branch to about 2^{15} words after.
2. No, it should be within a block of 2^{28} byte addresses where the PC supplies the upper 4 bits.
3. The target address is beyond the range where the bne instruction can go.
4. 如下表：

Address16	MIPS assembly instructions
FC80 0000	L1 : add \$s0, \$s1, \$s0
...	...
FC8A 0000	bne \$s0, \$s1, L2
...	j L1
	L2: lw \$s0, 12(\$s1)

5. 如下表：

Address16	Machine instructions
FC80 0000	000000 10000 10001 10000 000000 000000
...	...
FC8A 0000	000101 10000 10001 0000000000000001
FC8A 0004	000010 11001000000000000000000000
FC8A 0008	100011 10001 10000 0000000000001100
...	...

遠距離跳躍：

其實若是在實際執行上，寫出超過範圍之組合語言，組譯時依然會過且能執行，因為組譯器會自動將它轉換成可執行的替代碼，例如：

```
beq $s0, $s1, L1
xxx
```

L1:

```
beq $s0, $s1, L2
j L1
```

L2: xxx

...

L1:

練習：Given your understanding of PC-relative addressing, explain why an assembler might have problems directly implementing the branch instruction in the following code sequence:

here: beq \$s0, \$s2, there

...

there: add \$s0, \$s0, \$s0

Show how the assembler might rewrite this code sequence to solve these problems.

here: beq \$s0, \$s2, skip
j there

skip: ...

there: add \$s0, \$s0, \$s0

例(42)：Assume that the opcodes of the add, addi and lw instructions are respectively 000000, 001000, and 100011 in binary notation. Translate the following machine codes into corresponding assembly codes.

1. 000000 01001 01010 01000 00000 100000
2. 001000 10110 10101 111111111001110
3. 10001101001 01000 0000010010110000

1. add \$t0, \$t1, \$t2
2. addi \$s5, \$s6, -50
3. lw \$t0, 1200(\$t1)

例(32)：(翻譯)：What are the assembly codes that represent the machine codes stored in these locations?

Address	Content	Address	Content
80000H	02H	80004H	ADH
80001H	49H	80005H	52H
80002H	40H	80006H	00H
80003H	20H	80007H	C8H

Address	Content
80000H-80003H	add \$s0, \$s2, \$t1
80004H-80007H	sw \$s2, 200(\$t2)

練習：For the binary entries above, what instruction do they represent? What type instruction do the binary entries above represent?

a.	1010 1110 0000 1011 0000 0000 0000 0100
b.	1000 1101 0000 1000 0000 0000 0100 0000

a.	sw \$t3, 4(\$s0)	I type
b.	lw \$t0, 64(\$t0)	I type

練習：For the instructions above, show the hexadecimal representation of these instructions. What type instruction do the instructions above represent?

a.	add \$t0, \$t0, \$zero
b.	lw \$s1, 4(\$s3)

a.	0x0100 4020	R type
b.	0x8E69 0004	I type

練習：In the following problems, the data table contains the values of various fields of MIPS instructions.

a.	OP=0, rs=1, rt=2, rd=3, shamt=0, funct=32
b.	OP=0x2B, rs=0x10, rt=0x5, const=0x4

What is the MIPS assembly instruction described above? What type instruction do the instructions above represent? What is the binary representation of the instructions above?

a.	add \$v1, \$a1, \$v0	R type	0x0022 1820
b.	sw \$a1, 4(\$s0)	I type	0xAE05 0004

練習：What is the MIPS assembly language instruction corresponding to the machine instruction with the value 0000 0000_(hex)?

sll \$s0, \$s0, 0

練習：The data table contains various modifications that could be made to the MIPS instruction set architecture. If the instruction set of the MIPS processor is modified, the instruction format must also be changed. For each of the suggested changes below:

a.	8 registers
b.	10 bits immediate constants

- Show the size of the bit fields of an R type format instruction. What is the total number of bits needed for each instruction?
- Show the size of the bit fields of an I type format instruction. What is the total number of bits needed for each instruction?
- Why could the suggested change in the table above decrease the size of a MIPS assembly program? Why could the suggested change in the table above increase the size of a MIPS assembly program?

1. 如下表

	type	opcode	rs	rt	rd	shamt	funct	
a.	R type	6	3	3	3	5	6	Total bits= 26
b.	R type	6	5	5	5	5	6	Total bits=32

2. 如下表

	type	opcode	rs	rt	immed	
a.	I type	6	3	3	16	Total bits= 28
b.	I type	6	5	5	10	Total bits=26

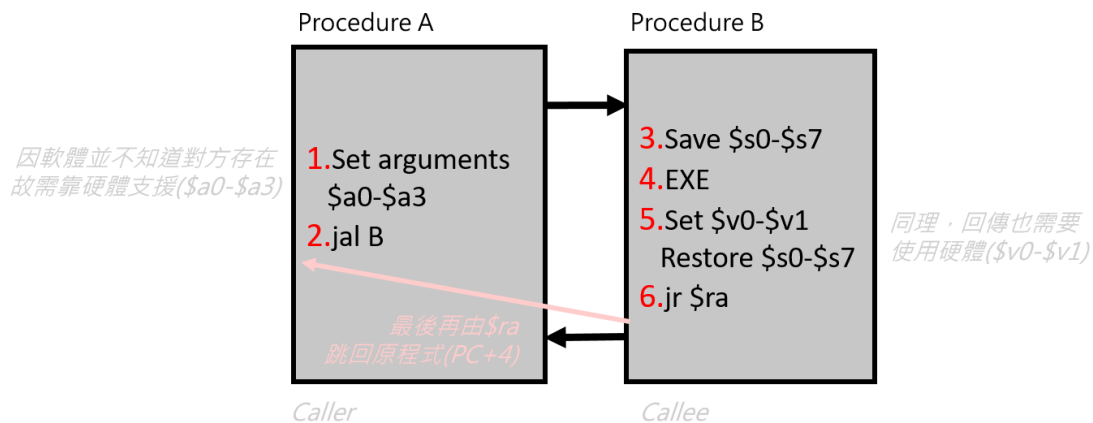
3. 如下表

a.	Less register -> less bits per instruction -> could reduce code size Less register -> more register spills -> more instructions -> could increase code size
b.	Smaller constants -> more lui instructions -> could increase code size Smaller constants -> less bits per instruction -> smaller code size

重點十三：程序呼叫

程序呼叫的六個步驟：

1. 將引數放在程序可以存取的地方(\$a0-\$a3)
2. 將控制權轉移給程序(jal B)
3. 取得程序所需的儲存資源(\$s0-\$s7)
4. 執行所指定的工作
5. 將所得結果放在呼叫程式可以存取得到的地方(\$s0-\$s7、\$v0-\$v1)
6. 將控制權交回給呼叫程序(jr \$ra)



例(14)：In the execution of a procedure, the program must first put the parameters in a place where the procedure can access them, and (after performing the desired task) return control to the point of origin. How can these two steps be achieved in computer hardware for a MIPS machine?

Register are the fastest place to hold data in a computer, so MIPS hardware provides:

\$a1-\$a3 : four argument registers in which to pass parameters.

\$ra : one return address register to return to the point of origin

例(39)：

1. The MIPS processor has an instruction jal. Describe the conditions in which you need to use the jal instruction when writing a MIPS assembly program.
2. Describe the detailed hardware operations for the MIPS processor to execute a jal instruction.

1. The jal instruction is used when a procedure call is required in a MIPS assembly language.
2. It jumps to an address and simultaneously saves the address of the following instruction in a register (\$ra in MIPS)

例(27)：Multiple choices-please circle all correct answers

The jr instruction in the MIPS architecture is typically used for which of the following C constructs?

1. A or loop
2. A switch statement
3. A nested If-then-else statement
4. A function

2、4

例(71) : Which of the following is (are) true?

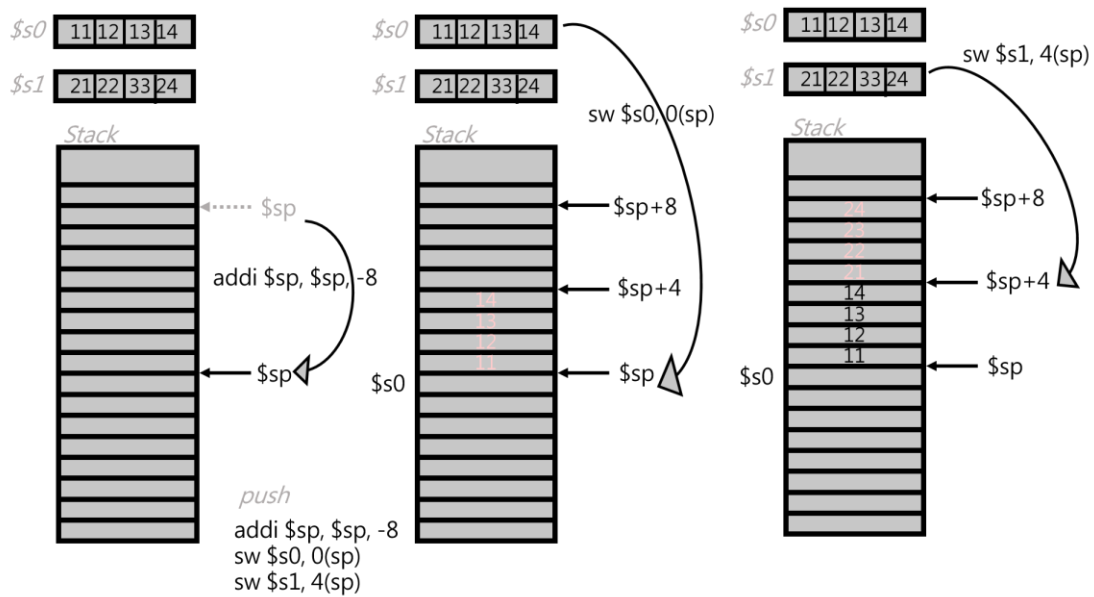
1. ISA is an abstraction which is the interface between the hardware and the low-level software (assembly instructions). This abstract interface enables different implementations of the same ISA to run identical software.
2. A caller is the program that is called by the procedure which gives the call.
3. A basic block is a sequence of instructions with branch at the beginning and at the end
4. A register file is a large memory for storing files

1

程序呼叫時的 Stack 存取

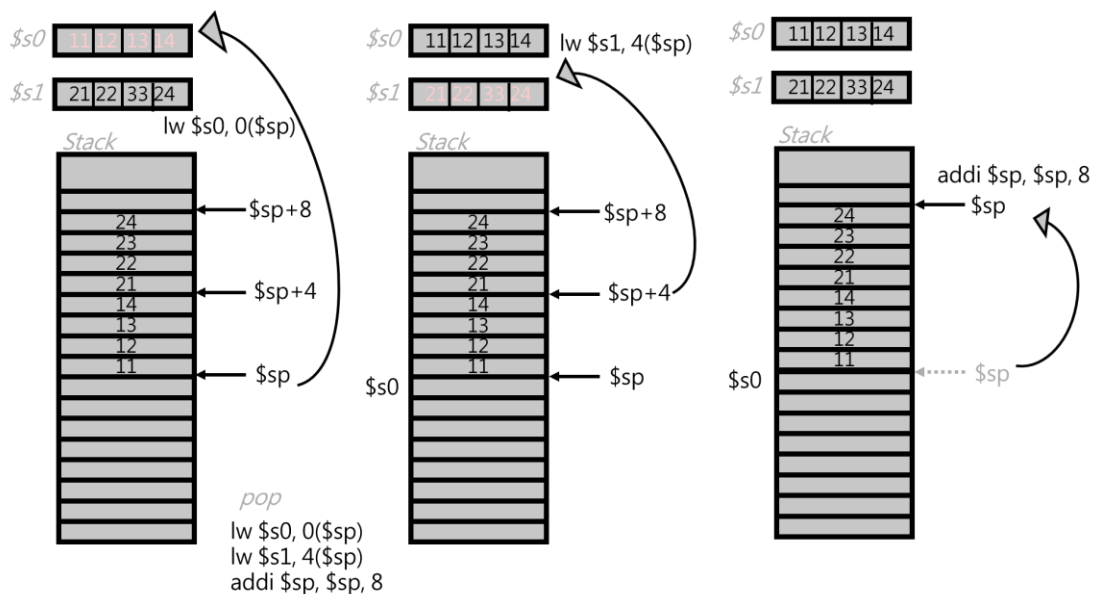
第 3 步驟：為了避免 Callee 蓋掉 Caller 參數，需要 Save register(Push : \$s0-\$7) :

`addi $sp, $sp, -8      sw $s0, 0(sp)      sw $s1, 4(sp)`



第 5 步驟：回復 Caller 的參數，需要 Restore register(Pop : \$s0-\$7) :

`lw $s0, 0($sp)    lw $s1, 4($sp)    addi $sp, $sp, 8`



練習：Convert the C function below to MIPS assembly language. (assume that arguments g, h, i, j in \$a0, \$a1, \$a2, \$a3 and f in \$s0)

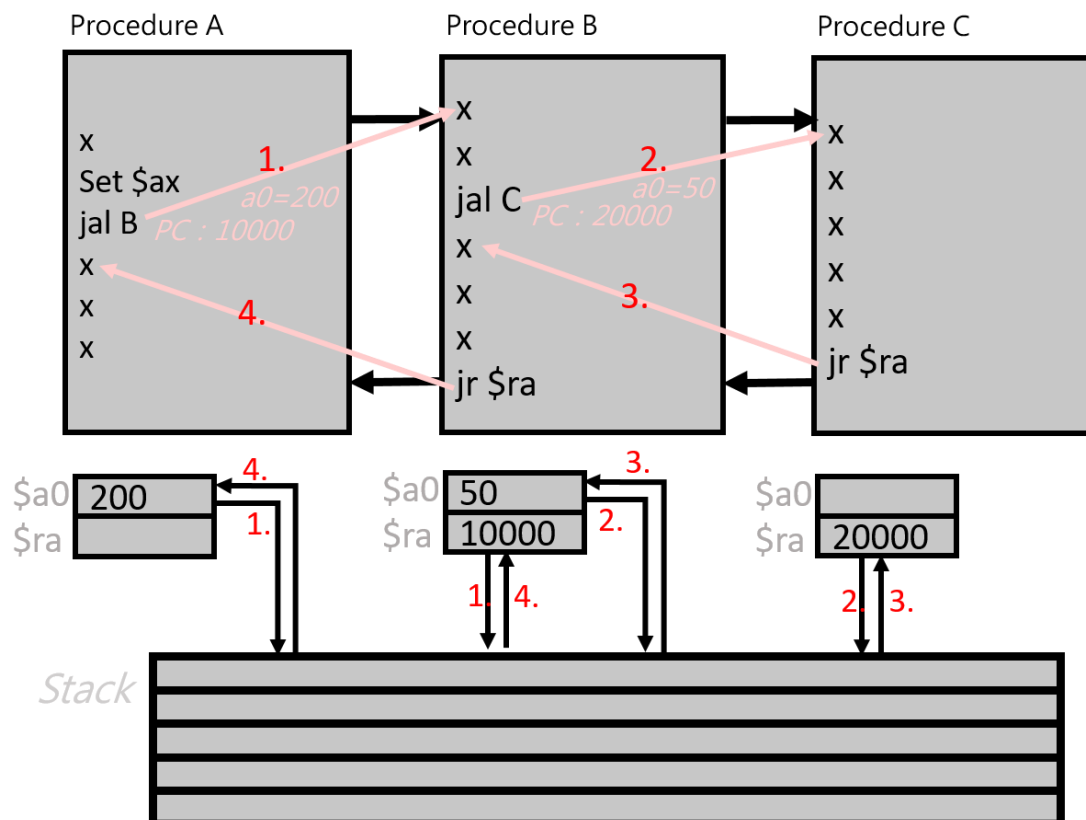
```
Int leaf_example (int g, h, i, j)
{
    int f;
    f=(g+h)-(i+j);
    return f;}

```

<i>Leaf_example:</i>	
<i>addi \$sp, \$sp, -4</i> <i>sw \$s0, 0(\$sp)</i>	Save \$s0 on stack
<i>add \$t0, \$a0, \$a1</i> <i>add \$t1, \$a2, \$a3</i> <i>sub \$s0, \$t0, \$t1</i>	Procedure body
<i>add \$v0, \$s0, \$zero</i>	Result
<i>lw \$s0, 0(\$sp)</i> <i>addi \$sp, \$sp, 4</i>	Restore \$s0
<i>jr \$ra</i>	Return

\$ra、\$ax 之存取

當多層次 Calling 出現時，很可能因為有多個 \$ra 與 \$ax，而複寫掉之前之參數，故每次呼叫前、後，都需要進行資料存取、保留的動作



練習：Convert the C function below to MIPS assembly language.

```
int fact(int n)
{
    if(n<1) return 1;
    else return (n*fact(n-1));
}
```

fact:	L1:
addi \$sp, \$sp, -8	addi \$a0, \$a0, -1
sw \$ra, 4(\$sp)	jal fact
sw \$a0, 0(\$sp)	lw \$a0, 0(\$sp)
slti \$t0, \$a0, 1	lw \$ra, 4(\$sp)
beq \$t0, \$zero, L1	addi \$sp, \$sp, 8
addi \$v0, \$zero, 1	mul \$v0, \$a0, \$v0
addi \$sp, \$sp, 8	jr \$ra
jr \$ra	

例(57)：

1. What is procedure frame? Also, stack pointer and frame pointer are used to maintain procedure frame. Why does procedure frame require two pointers?
2. Procedure has to spill registers to memory (save and then restore). Caller must take care of \$ax series and \$tx series and callee must take care of \$ra and \$sx series. Following codes require correction for spilling registers. Correct the errors and state your reasons.

fact:	L1:
addi \$sp, \$sp, -4	addi \$a0, \$a0, -1
sw \$ra, 4(\$sp)	jal fact
slti \$t0, \$a0, 1	lw \$ra, 0(\$sp)
beq \$t0, \$zero, L1	addi \$sp, \$sp, 4
addi \$v0, \$zero, 1	mul \$v0, \$a0, \$v0
addi \$sp, \$sp, 4	jr \$ra
jr \$ra	

1. A procedure frame is the segment of stack containing a procedure's saved registers and local variables. A frame pointer is used to point to the location of the saved registers and local variables for a given procedure. A stack pointer might change during the procedure, and so references to local variable in memory might have different offsets depending on where they are in the procedure, making the procedure harder to understand. Alternatively, a frame pointer offers a stable base register within a procedure for local memory references.
2. Since procedure fact will recursively call itself and the argument in register \$a0 will still be used after the call has returned, the content of register \$a0 should be saved before the call has made.

fact:	L1:
addi \$sp, \$sp, -8	addi \$a0, \$a0, -1
sw \$ra, 4(\$sp)	jal fact
sw \$a0, 0(\$sp)	lw \$a0, 0(\$sp)
slti \$t0, \$a0, 1	lw \$ra, 4(\$sp)
beq \$t0, \$zero, L1	addi \$sp, \$sp, 8
addi \$v0, \$zero, 1	mul \$v0, \$a0, \$v0
addi \$sp, \$sp, 8	jr \$ra
jr \$ra	

練習：Convert the C function below to MIPS assembly language. Make sure that your assembly language code could be called from a standard C program (that is to say, make sure you follow the MIPS calling conventions).

```
unsigned int sum(unsigned int n)
{
    if(n==0) return 0;
    else return n+sum(n-1);
}
```

This machine has no delay slots. The stack grows downward (toward lower memory addresses).

<pre>sum addi \$sp, \$sp, -8 sw \$ra, 4(\$sp) sw \$a0, 0(\$sp) add \$v0, \$zero, \$zero beq \$a0, \$zero, \$zero addi \$a0, \$a0, -1 lw \$ra, 4(\$sp) lw \$a-, 0(\$sp)</pre>	<pre>return: addi \$sp, \$sp, 8 add \$v0, \$a0, \$v0 jr \$ra</pre>
--	---

<pre>sum: addi \$sp, \$sp, -8 sw \$ra, 4(\$sp) sw \$a0, 0(\$sp) bne \$a0, \$zero, L1 addi \$v0, \$zero, 1 addi \$sp, \$sp, 8 jr \$ra</pre>	<pre>L1: addi \$a0, \$a0, -1 jal fact lw \$a0, 0(\$sp) lw \$ra, 4(\$sp) addi \$sp, \$sp, 8 add \$v0, \$a0, \$v0 jr \$ra</pre>
---	---

重點十四：支援平行程式的同步指令

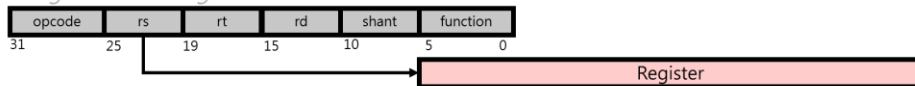
(省略，沒有考過)

重點十五：MIPS 定址模式

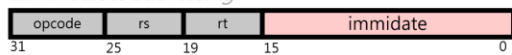
五、定址模式(Addressing Mode)

1. 暫存器定址模式(Register addressing)：運算元在暫存器中
2. 立即定址模式(Immediate addressing)：運算元即為包含在指令本身的 16 位元常數
3. 基底或位移定址模式(Base or displacement addressing)：運算元在記憶體中，而其記憶體位址的計算方式為將基底暫存器的內容加上指令中 16 位元的常數而得
4. 程式計數器相對定址模式(PC-relative addressing)：目的位址為程式計數器的內容加上指令中的常數
5. 虛擬直接定址模式(Pseudo-direct addressing)：跳躍的目的位址為指令的 26 位元與程式計數器高位元的結合

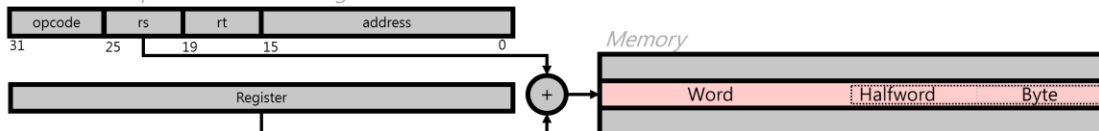
1. Register addressing



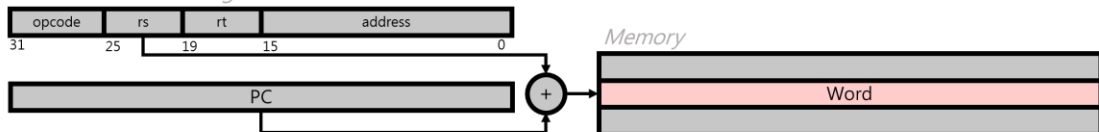
2. Immediate addressing



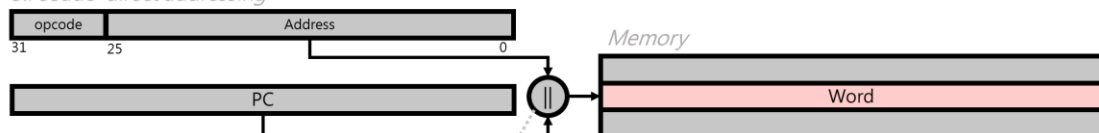
3. Base and displacement addressing



4. PC-relative addressing



5. Pseudo-direct addressing



符號" || " : Concatenation(合併)
例: "ABC" || "123" = "ABC123"

例(11)：Explain the following terms: Addressing modes?

指令取得運算元或計算目的位址的方法，稱為定址模式

例(59)：Explain the five MIPS addressing modes.

1. 暫存器定址模式(Register addressing)：運算元在暫存器中
2. 立即定址模式(Immediate addressing)：運算元即為包含在指令本身的 16 位元常數
3. 基底或位移定址模式(Base or displacement addressing)：運算元在記憶體中，而其記憶體位址的計算方式為將基底暫存器的內容加上指令中 16 位元的常數而得
4. 程式計數器相對定址模式(PC-relative addressing)：目的位址為程式計數器的內容加上指令中的常數
5. 虛擬直接定址模式(Pseudo-direct addressing)：跳躍的目的位址為指令的 26 位元與程式計數器高位元的結合

例(79)：Please draw the formats of five MIPS addressing modes.

見本節圖片

例(62)：(翻譯)：下列為何種定址模式、及其機器語言為何？

add \$t0, \$s1, \$s2、lw \$t0, 32(\$t1)

Register addressing 與 Base or displacement addressing

000000 01011 01010 10100 00000 100000

100011 01001 10000 00000 00000 110000

重點十七：RISC 與 CISC 的比較

RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
所有指令都有相同長度 (指令長度一致，擷取時速度較快)	指令的長短不一
只有少數幾種定址模式 (定址模式少擷取運算元速度較快)	支援多種定址模式
只有少數幾種指令模式 (指令格式少解碼較容易)	支援多種指令格式
算數指令的運算元只允許來自暫存器	算數指令的運算元可來自記憶體
記憶體中的資料需先載入至暫存器後才能處理	記憶體中的資料可以直接處理而不需事先載入暫存器

例(50)：The instruction encoding of RISCs uses fixed length instruction, please discuss its advantages and disadvantages.

Advantages: easy to fetch instruction

Disadvantage: poor memory utilization

例(77)：Compare the instruction-set architectures in RISC and CISC processors in terms of at least 5 important characteristics.

RISC	CISC
所有指令都有相同長度	指令的長短不一
只有少數幾種定址模式	支援多種定址模式
只有少數幾種指令模式	支援多種指令格式
算數指令的運算元只允許來自暫存器	算數指令的運算元可來自記憶體
記憶體中的資料需先載入至暫存器後才能處理	記憶體中的資料可以直接處理而不需事先載入暫存器

例(37)：(Multi choices question) Choose the correct description for a RISC processor:

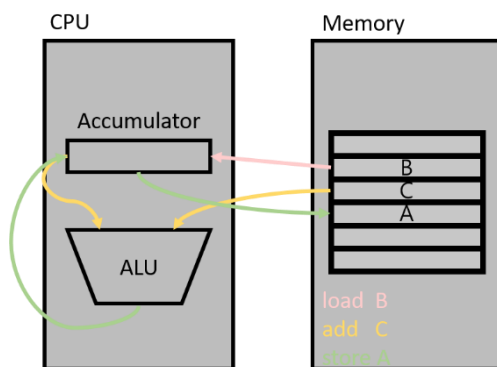
1. RISC processors have more specialized register rather than general purpose.
2. RISC processors have a very limited number of addressing modes.
3. RISC processors using 2-way pipelined timing (only two instructions can be in the pipe at any one time) do so in order to avoid two simultaneous memory accesses.
4. RISC processors use a fixed instruction length.

2、4；註：3.MIPS 可同時執行 5 個指令、ARM 可同時執行 3 個指令

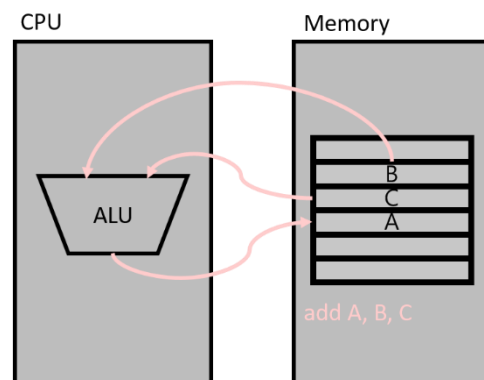
重點十八：不同型態的指令集

1. 累積器(Accumulator)：所有運算都發生在一個累積暫存器與記憶體位址之間
2. 記憶體-記憶體(Memory-Memory)：每個運算指令有 3 個運算元，而且這 3 個運算元都在記憶體中
3. 堆疊(Stack)：所有運算都發生在堆疊頂端。只能以 **push** 和 **pop** 指令存取記憶體，其他所有指令都是從堆疊中取走運算元，然後再把運算結果存回。這種做法使用到堆疊最頂端的兩個元素
4. 載入-儲存(Load-Store)：所有運算都發生在暫存器，而且每個指令都有 3 個運算元

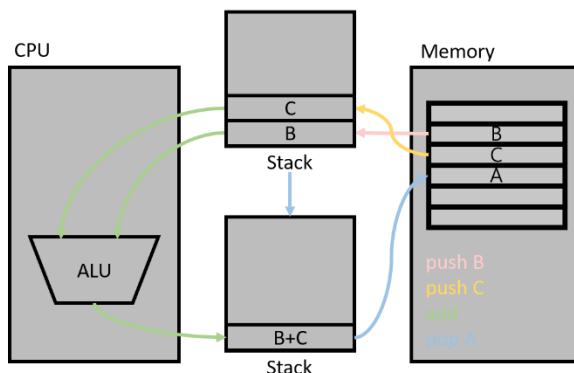
Accumulator (One operand)



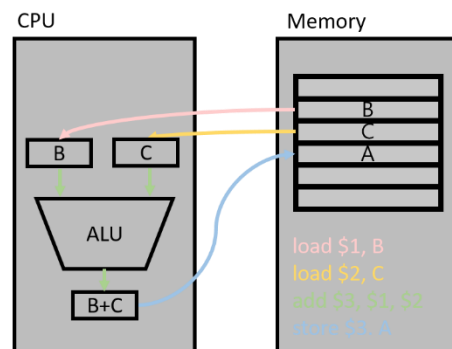
Memory-Memory (Three operand)



Stack (Zero operand)



Load-Store (Three operand)



假設：運算碼=1byte；記憶體位址=2bytes(也就使得 Reg=0.5bytes)；運算元=4bytes
以整數位元組為單位

A=B+C;

Style	IC	Code byte	Data byte
Accumulator	3	$(1+2) + (1+2) + (1+2) = 9$	$4+4+4=12$
M-M	1	$1+2+2+2=7$	$4+4+4=12$
Stack	4	$(1+2) + (1+2) + 1 + (1+2) = 10$	$4+4+0+4=12$
L/S	4	$4(1+0.5+2) + 4(1+0.5+2) + 3(1+0.5+0.5+0.5) + 4(1+0.5+2) = 15$	$4+4+0+4=12$

例(80) : There are 4 architecture styles: Accumulator, Memory-Memory, Load-Store, and Stack. Write the four code sequences for the C code $a=b+c-d$ using assembly language with minimum instruction counts in each architectural style. For each code sequence, calculate the instruction bytes fetched and memory-data bytes transferred (read or write). Assume:

1. The opcode is always 1 byte.
2. All memory addresses are 2 bytes
3. All data operands are 2 bytes.
4. All instructions are in an integral number of bytes in length.
5. The variables a, b, c, and d are initially in memory.
6. For load-store architecture, there are 16 general purpose registers.

ISA types	Accumulator	Mem-Mem	Stack	Load-Store
Code Sequence	load b add c sub d store a	add a, b, c sub a, a, d	push b push c add push d sub pop a	lw \$t0, b lw \$t1, c lw \$t2, d add \$t3, \$t0, \$t1 sub \$t3, \$t3, \$t2 sw \$t3, a
Instruction count	4	2	6	6
Instruction bytes	$3+3+3+3=12$	$7+7=14$	$3+3+1+3+1+3=14$	$4+4+4+3+3+4=22$
Memory Data Transfer	$2+2+2+2=8$	$6+6=12$	$2+2+0+2+0+2=8$	$2+2+2+0+0+2=8$

例(40) : The code below uses a three-operand instruction:

ADD C, A, B; $C=A+B$

In the space below, write three short programs that do exactly the same thing, one with two-operand instructions, one with one-operand instructions, and one with zero-operand instructions. For fitting the need of the instruction, please use register names R1, R2, etc.

Two operand instruction	One operand instruction	Zero operand instruction
Load R1, A Add R1, B Store R1, C	Load A Add B Store C	Push A Push B Add Pop C

例(25) :

1. For the “load-store” instruction set architecture processor, write down the assembly codes to compute $x+y-z$ and then stores the result back to C.
2. Suppose that all instruction operation codes are 6 bits, memory addresses are 24 bits (byte-addressable), and register addresses are 5 bits. What are the code sizes (in terms of bits) and the total memory accesses (in terms of bytes) for the codes in 1.?

1	2	
	Code size	Memory accesses
load \$s1, A	$6+5+24=35$	$8+4=12$
load \$s2, B	$6+5+24=35$	$8+4=12$
load \$s3, C	$6+5+24=35$	$8+4=12$
add \$t1, \$t1, \$s2	$6+5+5+5=21$	4
sub \$t2, \$t1, \$s3	$6+5+5+5=21$	4
store \$t2, C	$6+5+24=35$	$8+4=12$
Total	182 bits	56 bytes

註：長度 35 bits 指令若要對齊 word address，每個指令會佔用 $\lceil 35/32 \rceil = 2$ words = 8 bytes

重點十六：指令集的設計原則

1. 一致才會簡單：
 - (1) 讓所有指令保持單一大小
 - (2) 在算術指令中，一律要求 3 個運算元
 - (3) 讓不同指令格式中的暫存器欄位在同樣的位置
2. 愈小就愈快：MIPS 只有少量(32 個)暫存器，功率也較低
3. 使經常發生的事件快速：
 - (1) PC 相對定址法用於分支跳躍(因為有一半的指令跳躍距離不超過 16 條指令)
 - (2) 使用立即定址法以得到常數運算元
4. 好的設計需要有好的折衷方案：使用 3 種指令格式(R, I, J type)，而非 1 種

例(8)：To achieve “beq r1, r2, L1” in ARM’s assembly is done by cmp r1, r2, followed by beq L1.

Which of the following is (are) true?

1. cmp sets the comparison result in register r1.
2. cmp sets the comparison result in a condition code register.
3. beq L1 compares r1 and r2
4. cmp performs JUMP to L1.

2

例(7)：A MIPS conditional branch instruction is given in the form of “beq r1, r2, L1”. Which of the following is (are) true?

1. L1 is the absolute memory address of the branch target.
2. To form the binary machine code, the value of L1 is computed by the assembler when L1 is an external reference.
3. To form the binary machine code, the value of L1 is computed by the linker when L1 is an external reference.
4. When the instruction is executed, the binary value of L1 is directly used by the PC of the processor for branch operation.

3 ; 註：The value of L1 is computed by the assembler when L1 is an internal reference

例(29)：What are the four design principles advocated by Patterson and Hennessy in their textbooks? For each principle, give one example from the MIPS architecture that follows that principle.

1. 一致才會簡單：
 - (1) 讓所有指令保持單一大小
 - (2) 在算術指令中，一律要求 3 個運算元
 - (3) 讓不同指令格式中的暫存器欄位在同樣的位置
2. 愈小就愈快：MIPS 只有少量(32 個)暫存器，功率也較低
3. 使經常發生的事件快速：
 - (1) PC 相對定址法用於分支跳躍(因為有一半的指令跳躍距離不超過 16 條指令)
 - (2) 使用立即定址法以得到常數運算元
4. 好的設計需要有好的折衷方案：使用 3 種指令格式(R, I, J type)，而非 1 種

例(69)：One of the most important aspects of computer design is instruction set architecture (ISA) design, because it affects many aspects of the computer system including implementation of CPU and compiler.

1. Give the useful information that must be encoded in an instruction set architecture(ISA)
2. What are the principles of designing instruction set?

1. *Memory architecture*、*Registers*、*Instructions*、*Instruction format*、*Addressing mode*
2. 一致才會簡單、愈小就愈快、使經常發生的事件快速、好的設計需要有好的折衷方案

例(72)：Which of the following statements confirming to the design principle: simplicity favors regularity?

1. Keeping all instructions in a single size.
2. Always requiring three operands in arithmetic instruction
3. Keeping the register fields in the same place in each instruction format.
4. Having the same opcode field in the same place in each instruction format.

1、2、3；4 錯在不論好壞設計，opcode 位址都應該統一固定

練習：When designing memory system, it becomes useful to know the frequency of memory reads versus writes as well as the frequency of accesses for instructions versus data. Using the average instruction mix information for MIPS for the program SPEC2000int. (36% of all instructions for SPEC2000int are data access instructions,) find the following:

1. The percentage of all memory access (both data and instruction) that are for data.
2. The percentage of all memory accesses (both data and instruction) that are for reads. Assume that two-thirds of data transfers are loads.

Since 36% are data access instructions. Thus, for 100 instructions, there are 36 data accesses, and a total of 136 memory accesses (100 to read instructions and 36 to access data.)

1. The percentage of memory access that are for data = $36/136 = 26\%$
2. Assuming 2/3 of data transfers are loads, the percentage of memory accesses that are reads = $(100 + 36 \times 2/3) / 136 = 91\%$

重點十九：ARM 與 MIPS 的比較

	ARM	MIPS
<i>Data announced</i>	1985(第 2 個)	1985(第 1 個)
<i>Instruction size (bits)</i>	32	32
<i>Address space (size, model)</i>	32 bits, flat	32 bits, flat
<i>Data alignment</i>	Aligned	Aligned
<i>Data addressing modes</i>	9	3
<i>Integer registers</i>	15 GPR * 32 bits	32 GPR * 32 bits
<i>IO</i>	Memory mapped	Memory mapped

註：MIPS 管線上可同時執行 5 個指令、ARM 可同時執行 3 個

練習：Translate the ARM assembly code to MIPS assembly code. Assume that ARM registers r0, r1, r2 hold the same values as MIPS registers \$s0, \$s1, and \$s2, respectively. Use MIPS temporary registers (\$t0, etc.) where necessary.

a.	<pre> MOV r0, #10 LOOP: ADD r0, r1 SUBS r1, 1 BNE LOOP </pre>
b.	<pre> ROR r1, r2, #4 </pre>

a.	<i>addi \$s0, \$zero, 10</i> <i>LOOP: add \$s0, \$s0, \$s1</i> <i>addi \$s1, \$s1, -1</i> <i>bne \$s1, \$zero, LOOP</i>
b.	<i>sll \$s1, \$s2, 28</i> <i>srl \$t0, \$s2, 4</i> <i>or \$s1, \$s1, \$r0</i>

練習：

a.	<i>ADD r3, r2, r1, LSL #3</i> ; <i>r3=r2+(r1<<3)</i>
b.	<i>ADD r3, r2, r1, ROR #3</i> ; <i>r3=r2+(r1, rotated right 3 bits)</i>

1. Write the equivalent MIPS code for the ARM assembly code above.
2. If the register R1 had the constant value of 8, rewrite your MIPS code to minimize the number of MIPS assembly instructions need.
3. If the register R1 had the constant value of 0x0600 0000, rewrite your MIPS code to minimize the number of MIPS assembly instructions needed.

	1	2	3
a.	<i>sll \$t0, \$s1, 3</i> <i>add \$s3, \$s2, \$t0</i>	<i>addi \$s3, \$s2, 64</i>	<i>sll \$t0, \$s1, 3</i> <i>add \$s3, \$s2, \$t0</i>
b.	<i>sll \$s4, \$s1, 29</i> <i>srl \$t0, \$s1, 3</i> <i>or \$t0, \$t0, \$s4</i> <i>add \$s3, \$s2, \$t0</i>	<i>addi \$s3, \$s2, 1</i>	<i>srl \$t0, \$s1, 3</i> <i>add \$s3, \$s2, \$t0</i>